

```
In [1]: import pandas as pd
import yfinance as yf
from datetime import datetime
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sb
import statsmodels.api as sm
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsRegressor
```

```
In [2]: def get_stock_data(ticker, start_date, end_date):
# Download stock data
data = yf.download(ticker, start=start_date, end=end_date)
return data

# Set the ticker symbol, start date, and end date
ticker = "^BVSP" # BVSP
start_date = "2018-01-01"
end_date = datetime.today().strftime('%Y-%m-%d')

# Get the stock data
nse_data = get_stock_data(ticker, start_date, end_date)
nse_data.dropna(inplace=True)
nse_data
```

[*****100%*****] 1 of 1 completed

```
Out[2]:
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2018-01-02	76403.0	77909.0	76403.0	77891.0	77891.0	2880300
2018-01-03	77889.0	78414.0	77602.0	77995.0	77995.0	3142200
2018-01-04	77998.0	79135.0	77998.0	78647.0	78647.0	3486100
2018-01-05	78644.0	79071.0	78218.0	79071.0	79071.0	2747100
2018-01-08	79070.0	79395.0	78631.0	79379.0	79379.0	2550800
...	...	...	...	...	...	...
2023-08-15	116809.0	117697.0	116033.0	116171.0	116171.0	14636000
2023-08-16	116171.0	117338.0	115534.0	115592.0	115592.0	18862600
2023-08-17	115592.0	116610.0	114859.0	114982.0	114982.0	12747500
2023-08-18	114973.0	115729.0	114423.0	115409.0	115409.0	10601000
2023-08-21	115404.0	115425.0	114067.0	114429.0	114429.0	9794400

1398 rows × 6 columns

```
In [3]: # Calculate Stock High minus Low price (H-L)
nse_data['H-L'] = nse_data['High'] - nse_data['Low']

# Calculate Stock Open minus Close price (O-C)
nse_data['O-C'] = nse_data['Open'] - nse_data['Close']

# Calculate Stock price's seven days' moving average (7 DAYS MA)
nse_data['7 DAYS MA'] = nse_data['Close'].rolling(window=7).mean()

# Calculate Stock price's fourteen days' moving average (14 DAYS MA)
nse_data['14 DAYS MA'] = nse_data['Close'].rolling(window=14).mean()

# Calculate Stock price's twenty one days' moving average (21 DAYS MA)
nse_data['21 DAYS MA'] = nse_data['Close'].rolling(window=21).mean()

# Calculate Stock price's standard deviation for the past seven days (7 DAYS STD DEV)
nse_data['7 DAYS STD DEV'] = nse_data['Close'].rolling(window=7).std()

# Print the data
nse_data.dropna(inplace=True)
```

```
In [4]: # Calculate descriptive statistics
statistics = nse_data.describe()

# Print the statistics
statistics
```

Out[4]:

	Open	High	Low	Close	Adj Close	Volume	H-L	O-C	
count	1378.000000	1378.000000	1378.000000	1378.000000	1378.000000	1.378000e+03	1378.000000	1378.000000	1
mean	102898.248186	103891.377358	101905.232221	102918.216255	102918.216255	8.696005e+06	1986.145138	-19.96807	102
std	13870.969377	13794.617083	13911.961424	13864.907725	13864.907725	4.296711e+06	1164.498561	1564.89204	13
min	63604.000000	67604.000000	61691.000000	63570.000000	63570.000000	0.000000e+00	306.000000	-10057.00000	68
25%	95075.500000	96251.750000	94019.000000	95154.000000	95154.000000	4.814675e+06	1332.000000	-957.75000	95
50%	104585.500000	105490.000000	103709.500000	104531.000000	104531.000000	8.861750e+06	1767.500000	-48.50000	104
75%	113409.250000	114379.500000	112251.500000	113498.000000	113498.000000	1.169390e+07	2323.500000	787.25000	113
max	130776.000000	131190.000000	129526.000000	130776.000000	130776.000000	2.602930e+07	16615.000000	12520.00000	130

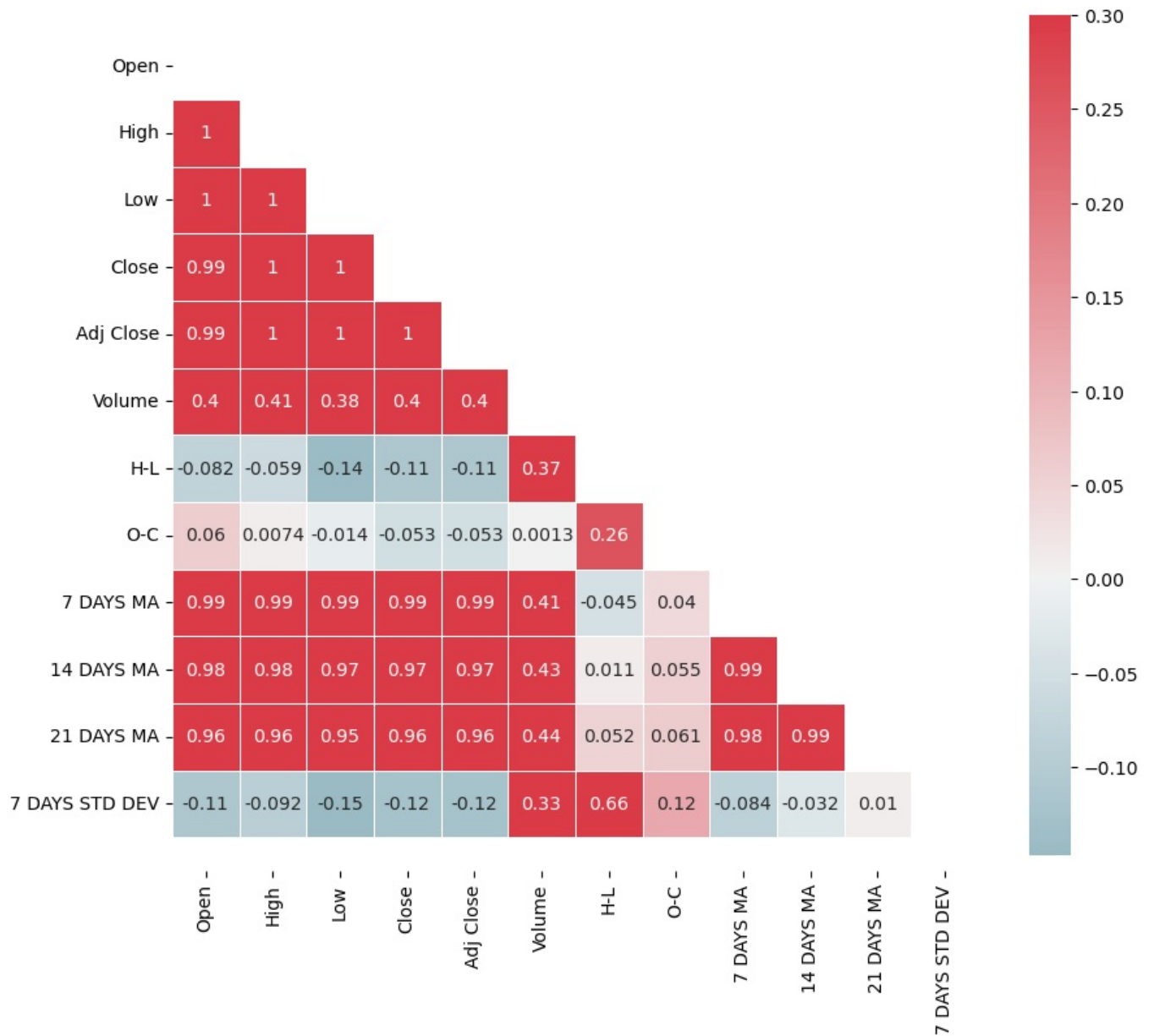
```
In [5]: # Calculate correlation matrix
correlation_matrix = nse_data.corr()

# Print the correlation matrix
correlation_matrix
```

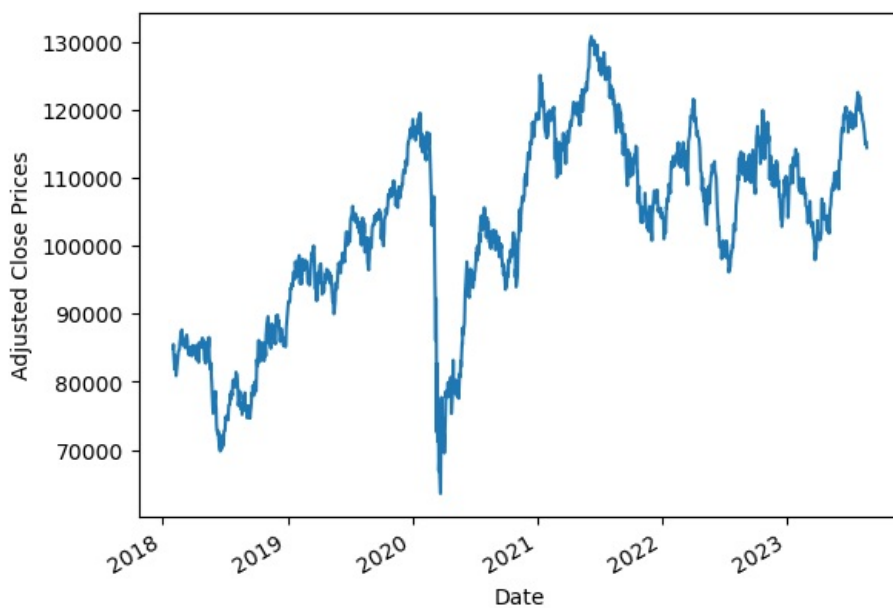
Out[5]:

	Open	High	Low	Close	Adj Close	Volume	H-L	O-C	7 DAYS MA	14 DAYS MA	21 DAYS MA	7 D STD
Open	1.000000	0.997546	0.995991	0.993633	0.993633	0.395736	-0.081947	0.060282	0.993276	0.979388	0.962307	-0.110726
High	0.997546	1.000000	0.996503	0.997144	0.997144	0.413110	-0.058988	0.007425	0.993435	0.980877	0.964942	-0.092240
Low	0.995991	0.996503	1.000000	0.998019	0.998019	0.378796	-0.142196	-0.014107	0.988787	0.971643	0.952457	-0.146568
Close	0.993633	0.997144	0.998019	1.000000	1.000000	0.395762	-0.110932	-0.052559	0.989177	0.973658	0.955788	-0.124397
Adj Close	0.993633	0.997144	0.998019	1.000000	1.000000	0.395762	-0.110932	-0.052559	0.989177	0.973658	0.955788	-0.124397
Volume	0.395736	0.413110	0.378796	0.395762	0.395762	1.000000	0.368320	0.001301	0.409551	0.425800	0.438779	0.329071
H-L	-0.081947	-0.058988	-0.142196	-0.110932	-0.110932	0.368320	1.000000	0.256489	-0.044575	0.011469	0.051921	0.658346
O-C	0.060282	0.007425	-0.014107	-0.052559	-0.052559	0.001301	0.256489	1.000000	0.040166	0.054557	0.061481	0.120692
7 DAYS MA	0.993276	0.993435	0.988787	0.989177	0.989177	0.409551	-0.044575	0.040166	1.000000	0.992685	0.978615	-0.083991
14 DAYS MA	0.979388	0.980877	0.971643	0.973658	0.973658	0.425800	0.011469	0.054557	0.992685	1.000000	0.994632	-0.032259
21 DAYS MA	0.962307	0.964942	0.952457	0.955788	0.955788	0.438779	0.051921	0.061481	0.978615	0.994632	1.000000	0.010104
7 DAYS STD DEV	-0.110726	-0.092240	-0.146568	-0.124397	-0.124397	0.329071	0.658346	0.120692	-0.083991	-0.032259	0.010104	1.000000

```
In [6]: f, ax = plt.subplots(figsize=(10, 12))
corr = nse_data.corr()
mask = np.triu(np.ones_like(corr, dtype=bool)) # Use bool directly
cmap = sb.diverging_palette(220, 10, as_cmap=True)
sb.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0, annot=True,
           square=True, linewidths=.5, cbar_kws={"shrink": .7})
bottom, top = ax.get_ylim()
ax.set_ylim(bottom + 0.5, top - 0.5)
plt.show()
```



```
In [7]: nse_data['Adj Close'].plot()
plt.ylabel("Adjusted Close Prices")
plt.show()
```



Linear Regression

```
In [8]: # Select the "Adj Close" and "Date" columns from the stock data
adj_close = nse_data['Adj Close']
high = nse_data['High']
```

```
# Create a DataFrame with "Adj Close" and "Date" columns
data = pd.DataFrame({'Adj Close': adj_close, 'High': high})

# Add a constant term to the independent variable
data = sm.add_constant(data)

# Fit the linear regression model
model = sm.OLS(data['Adj Close'], data[['const', 'High']]).fit()

# Print the regression results
print(model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  Adj Close    R-squared:                  0.994
Model:                            OLS        Adj. R-squared:              0.994
Method:                 Least Squares    F-statistic:                2.398e+05
Date:                Tue, 22 Aug 2023    Prob (F-statistic):          0.00
Time:                  22:31:08    Log-Likelihood:             -11537.
No. Observations:                1378    AIC:                        2.308e+04
Df Residuals:                    1376    BIC:                        2.309e+04
Df Model:                            1
Covariance Type:                nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const      -1204.2910     214.472     -5.615     0.000    -1625.019    -783.563
High         1.0022         0.002    489.740     0.000         0.998         1.006
=====
Omnibus:                 1192.916    Durbin-Watson:              1.722
Prob(Omnibus):              0.000    Jarque-Bera (JB):            52291.621
Skew:                   -3.804    Prob(JB):                     0.00
Kurtosis:                 32.204    Cond. No.                     7.97e+05
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 7.97e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Multiple Regression

```
In [9]: # Create a DataFrame with the selected columns
data = nse_data[['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']]

# Add a constant term to the independent variables
data = sm.add_constant(data)

# Fit the multiple regression model
model = sm.OLS(data['Adj Close'], data[['const', 'Open', 'High', 'Low', 'Close', 'Volume']]).fit()

# Print the regression results
print(model.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          Adj Close   R-squared:                1.000
Model:                  OLS         Adj. R-squared:           1.000
Method:                 Least Squares   F-statistic:              8.434e+28
Date:                   Tue, 22 Aug 2023   Prob (F-statistic):       0.00
Time:                   22:31:08         Log-Likelihood:           26925.
No. Observations:       1378           AIC:                     -5.384e+04
Df Residuals:           1372           BIC:                     -5.381e+04
Df Model:                5
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	6.139e-11	1.77e-10	0.346	0.729	-2.86e-10	4.09e-10
Open	-5.218e-15	3.01e-14	-0.173	0.862	-6.43e-14	5.39e-14
High	-1.61e-15	3.89e-14	-0.041	0.967	-7.79e-14	7.46e-14
Low	1.998e-15	3.5e-14	0.057	0.954	-6.67e-14	7.07e-14
Close	1.0000	3.63e-14	2.75e+13	0.000	1.000	1.000
Volume	-4.7e-17	6.11e-18	-7.698	0.000	-5.9e-17	-3.5e-17

```

=====
Omnibus:                26.778   Durbin-Watson:           0.027
Prob(Omnibus):           0.000   Jarque-Bera (JB):         16.402
Skew:                    0.106   Prob(JB):                 0.000274
Kurtosis:                2.509   Cond. No.                 8.06e+07
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 8.06e+07. This might indicate that there are strong multicollinearity or other numerical problems.

Random Forest

```

In [10]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

```

```

In [11]: x_train_rf, x_test_rf, y_train_rf, y_test_rf = train_test_split(x, y, test_size=0.20, random_state=0)

```

```

In [12]: scale = StandardScaler()
x_train_rf = scale.fit_transform(x_train_rf)
x_test_rf = scale.transform(x_test_rf)

```

```

In [13]: grid_rf = {
    'n_estimators': [20, 50, 100, 500, 1000],
    'max_depth': np.arange(1, 15, 1),
    'min_samples_split': [2, 10, 9],
    'min_samples_leaf': np.arange(1, 15, 2, dtype=int),
    'bootstrap': [True, False],
    'random_state': [1, 2, 30, 42]
}

# Instantiate the random forest model
model = RandomForestRegressor()

rscv = RandomizedSearchCV(estimator=model, param_distributions=grid_rf, cv=3, n_jobs=-1, verbose=2, n_iter=10,
rscv_fit = rscv.fit(x_train_rf, y_train_rf)

# Get the best parameters and best model
best_params = rscv_fit.best_params_
best_model = rscv_fit.best_estimator_

print(best_params)
print(best_model)

```

Fitting 3 folds for each of 10 candidates, totalling 30 fits

```
{'random_state': 2, 'n_estimators': 500, 'min_samples_split': 10, 'min_samples_leaf': 5, 'max_depth': 6, 'bootstrap': True}
```

```
RandomForestRegressor(max_depth=6, min_samples_leaf=5, min_samples_split=10,
n_estimators=500, random_state=2)
```

```

In [14]: model = RandomForestRegressor(n_estimators=314, random_state=42, min_samples_split=4, min_samples_leaf=3, max_d
model.fit(x_train_rf, y_train_rf)
predict = model.predict(x_test_rf)
print(predict)

```

118755.39715559	100282.54313316	94666.81922278	83842.457884
95912.66327125	87914.73109737	85718.17119555	114802.33622217
116483.78699707	105838.80452638	120239.84069827	126525.81736105
104853.19577807	114979.24408765	109764.72996178	78006.06999476
108466.25728573	109077.61654937	109891.23442271	88636.33024345
112915.2790238	111720.10426639	96708.67408654	108978.9753218
104303.38471027	95660.87331542	111970.79670527	113362.04781241
104746.01483764	117871.5884908	121444.76713239	94307.31178287
122946.30068393	110240.25076093	112765.23421596	114400.96650008
114290.34147354	81372.06545677	110570.53153444	108377.49126915
124544.65295103	120436.68966508	112750.21456432	100772.67821112
87893.98110243	108987.89595293	85525.20685184	74913.15909158
104357.79768419	102940.17997387	107379.49482927	105898.19134268
84825.07891266	88936.72690222	121778.97514225	76197.82893388
98612.39082637	95030.40899166	112414.52643407	95581.35162255
74593.54569786	109925.58560767	119394.36372759	85274.14666857
76253.21173642	113526.0103118	108184.56212819	126198.97756581
87967.53167951	104657.6929548	114111.74588646	82748.32095535
119236.85623053	122887.59571654	83251.24389217	107719.54036854
85606.43480652	94651.08643274	102241.97146789	112671.48102695
119242.96072957	85684.06668422	85760.95740282	100557.88239929
110480.39091081	120032.70219449	118403.94109846	98017.51797571
109733.24131175	93616.50597513	77874.44956604	78629.03264113
107346.67652383	109293.53187815	100278.34717161	118395.15546448
117473.71190453	88193.01224518	85530.48221018	113781.94368061
96554.53357362	120682.83767509	106976.4878411	83804.96765345
114636.39994945	115153.79123798	110035.7835228	80760.36796612
91773.72458275	102597.33107603	96216.04099667	114151.78892967
79673.11444377	112301.19046964	122035.54912882	78132.49977505
106357.6373071	115942.79846888	102274.98155913	97905.60841447
74690.38534812	85408.29820739	102115.57179774	110220.68783262
120885.01716126	125454.61657028	95995.78147846	123196.76667924
98796.37691404	85483.65418379	82038.12774331	85799.69747369
67641.60144702	85210.16903464	97467.3886565	119725.57030531
93018.71628639	91684.37981834	102069.25172712	122880.08732709
74694.95251398	102978.40508742	111088.54426615	101000.36272059
98803.4417078	85332.78291675	101511.4702165	111554.37018829
85492.91616414	114474.69767052	116425.89425444	107217.42695126
101229.55306785	107875.72479906	110951.16487557	86437.82738524
112297.71305204	109916.03121984	112733.06838851	104375.96620229
105999.58946172	94440.86711971	110992.39347565	114640.03344581
81240.41415441	125745.93614661	108791.00692243	113114.72225278
110126.13193356	110336.3049571	86574.46204554	96198.71768128
118083.93579023	98280.33474976	126769.758765	99791.96367359
102239.97508102	120189.50279513	99606.94562594	117686.89927745
79370.62460452	107221.41732014	120202.93340629	98667.37090606
84284.09445088	101976.39113152	121701.56313248	110331.28488847
113452.68150281	117530.38265515	101239.91722086	116674.10411054
76152.07647377	108459.92415552	110044.31444359	97643.6008288
103914.20963595	95377.82864923	79043.41252551	120017.77242626
101914.29895899	84523.80181087	77729.07094946	106486.0468192
70614.59632563	116241.79755023	117518.47620874	84670.2901575
82721.49201363	89854.18604015	75542.67958836	83000.09122251
114803.72953807	106531.74941729	113139.1334928	108370.90791353
72290.53861097	119072.21495882	117896.58877044	86852.42428815
93656.25364852	76767.12033693	121767.99322274	85479.9884877
83621.71470448	94664.48682338	105714.63246075	103841.57995388
101883.20836107	115524.00127351	86453.2690557	118893.0988432
109422.60205467	97302.47675578	78971.49394548	115338.92364661
119930.73268146	103699.42326549	107675.10458038	101821.09993136
112513.87390721	98027.18870288	112858.90602183	109923.61045349
121344.35447269	111082.00893875	84914.55034144	96953.25659966
93817.3262955	112571.88304396	119196.98063471	119744.11273403
111356.06296046	103903.5364775	102287.43176685	108323.15593848
94344.9448208	97399.07260662	100599.07151233	115371.12277274
107530.98100603	106807.56749422	102947.10104353	95348.95064902
108188.42917744	80922.65590465	104105.11121311	118499.31046361
95989.13806525	105938.34011682	101256.27293692	97479.38077308
95347.53186932	96108.79975417	121902.78147962	97870.54255248]

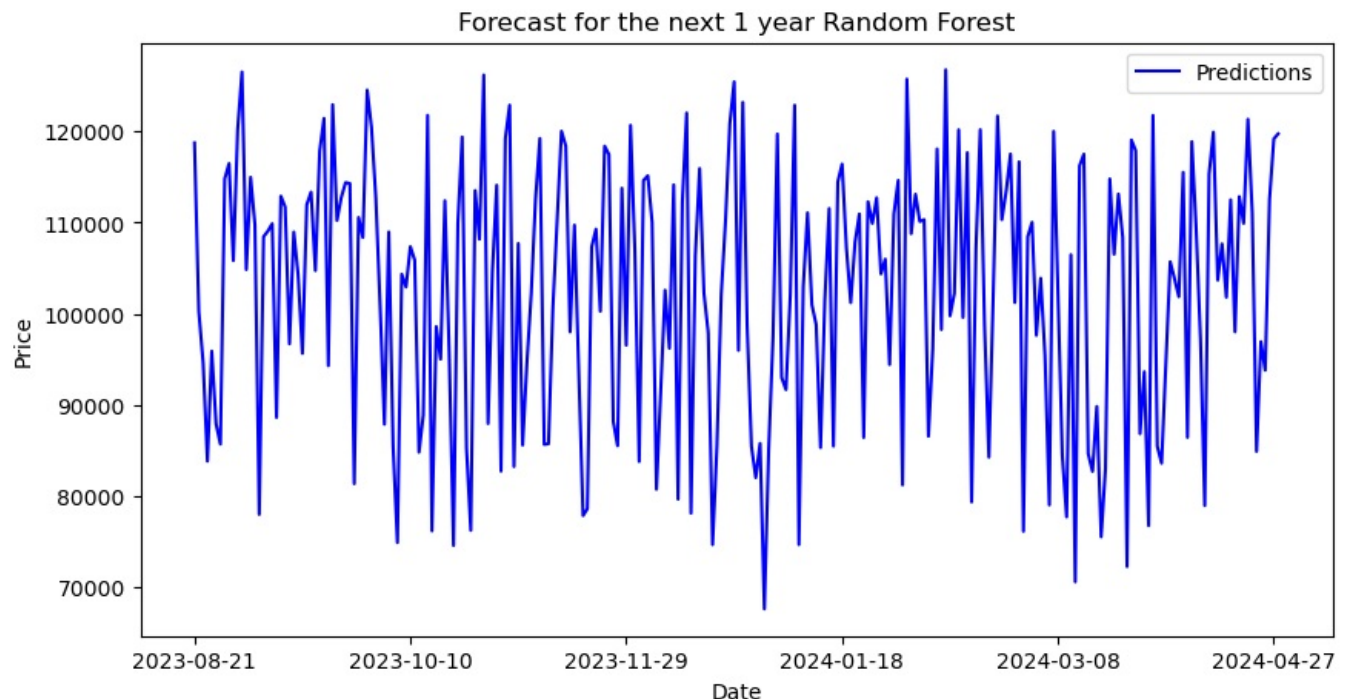
```
In [15]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_rf, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_rf, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_rf, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_rf, predict), 4))
print(f'Train Score : {model.score(x_train_rf, y_train_rf) * 100:.2f}% and Test Score : {model.score(x_test_rf,
errors = abs(predict - y_test_rf)
mape = 100 * (errors / y_test_rf)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 45.3215
Mean Squared Error: 9787.1136
Root Mean Squared Error: 98.9298
(R^2) Score: 0.9999
Train Score : 99.99% and Test Score : 99.99% using Random Tree Regressor.
Accuracy: 99.95 %.

```
In [16]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_rf_BVSP.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_rf_BVSP.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_rf_BVSP.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_rf_BVSP.csv")
```

```
In [17]: oneyear_df_pred = pd.read_csv("one-year-predictions_rf_BVSP.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Random Forest", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions
Date
2023-12-31 67641.601447
Sell price and date
Predictions
Date
2024-02-11 126769.758765



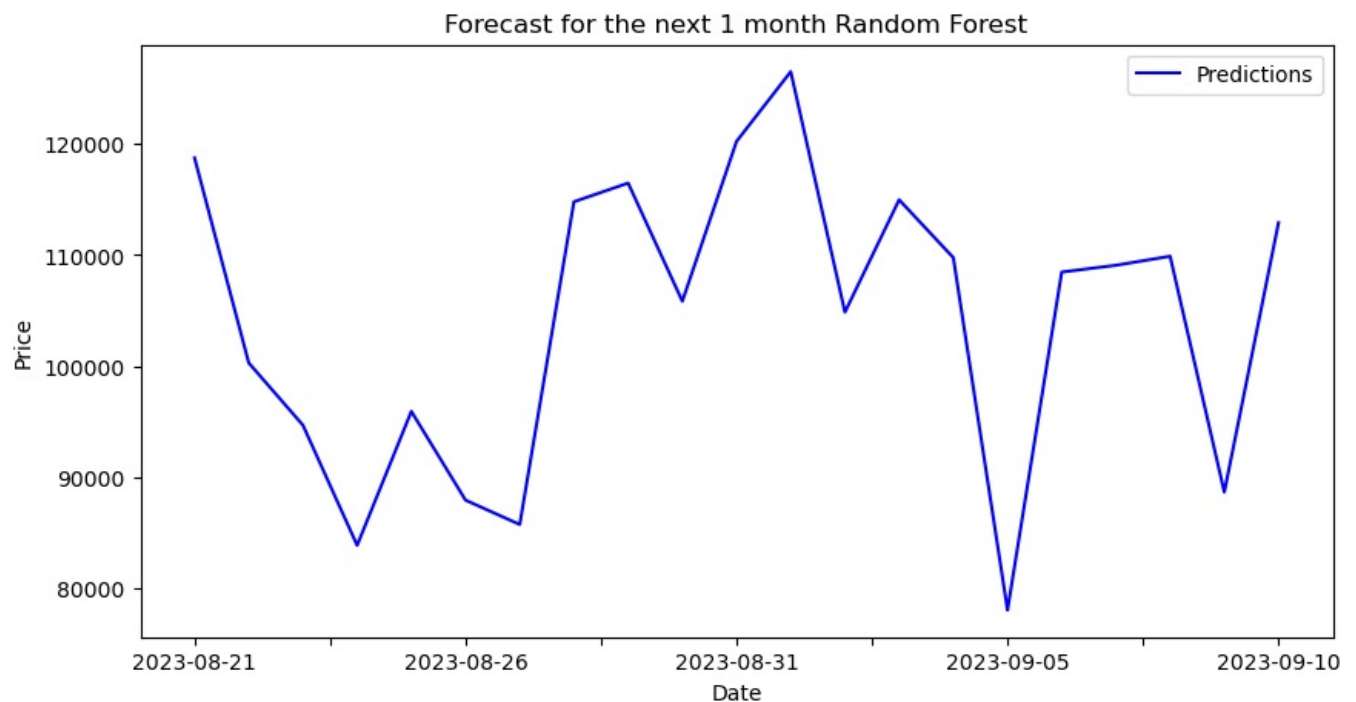
```
In [18]: onemonth_df_pred = pd.read_csv("one-month-predictions_rf_BVSP.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Random Forest", colo
plt.xlabel("Date")
```

```
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-09-05 78006.069995
Sell price and date

Predictions
Date
2023-09-01 126525.817361

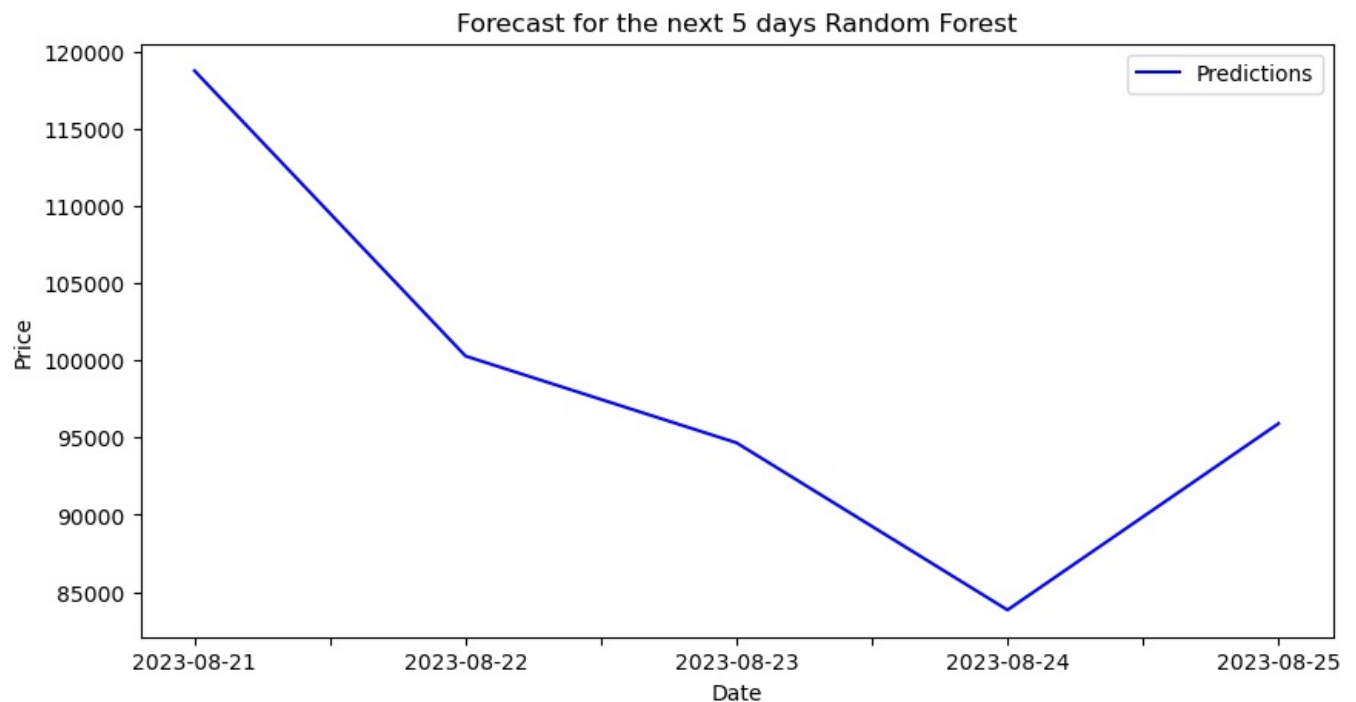


```
In [19]: fivedays_df_pred = pd.read_csv("five-days-predictions_rf_BVSP.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Random Forest", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-08-24 83842.457884
Sell price and date

Predictions
Date
2023-08-21 118755.397156



SVM

```
In [20]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [21]: x_train_svm, x_test_svm, y_train_svm, y_test_svm = train_test_split(x, y, test_size=0.20, random_state=0)

In [22]: scale = StandardScaler()
x_train_svm = scale.fit_transform(x_train_svm)
x_test_svm = scale.transform(x_test_svm)

In [23]: # Define the parameter grid for hyperparameter tuning
param_grid = {
    'C': [0.1, 1.0, 10.0],
    'gamma': [0.1, 1.0, 'scale'],
    'kernel': ['linear', 'rbf']
}

# Instantiate the SVM model
svm_model = SVR()

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=svm_model, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train_svm, y_train_svm)

# Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)
```

```
{'C': 10.0, 'gamma': 0.1, 'kernel': 'linear'}  
SVR(C=10.0, gamma=0.1, kernel='linear')
```

```
In [24]: # Retrain the best model on the full training set  
best_model.fit(x_train_svm, y_train_svm)  
  
# Make predictions  
predict = best_model.predict(x_test_svm)  
predict
```

```
Out[24]: array([[117991.40955108, 100622.95526871, 93413.39749234, 84525.85150348,  
96070.846686 , 88285.63165284, 85645.26214631, 114648.33510619,  
116904.97692567, 105210.95153643, 119909.40134025, 127273.52974912,  
105230.20945596, 114227.77453719, 110093.4543122 , 77728.48993399,  
107644.20273054, 108703.69628831, 110537.75839522, 87812.1501685 ,  
113177.04759969, 111814.74486358, 94853.3597624 , 108475.45728348,  
103555.52688376, 96602.43668236, 111978.81011332, 113283.07808913,  
104685.98087097, 118000.67366436, 121005.28851796, 94422.6565401 ,  
123247.10962706, 111704.72937898, 113083.25389536, 115113.72233321,  
113696.15618295, 81442.99830706, 110706.64102197, 108951.69191703,  
123722.35709481, 119882.63214913, 112148.84765078, 101445.12835011,  
87883.86137856, 109558.60462723, 85607.70280837, 74834.96687199,  
104652.74239906, 102349.81993975, 107008.53638741, 106284.7672879 ,  
85149.30872284, 88854.67283479, 121661.45650013, 76974.31489027,  
98054.59716285, 95129.23672898, 111735.44172596, 95978.01832214,  
75017.41717175, 109858.69730344, 119382.35324898, 85425.53683886,  
77057.31106016, 113427.0224224 , 108254.9788163 , 126309.84546775,  
87514.85137468, 104564.8799489 , 113727.75469616, 82988.62181958,  
118341.74971172, 123427.4444847 , 83626.01196822, 108357.66531673,  
85617.77896513, 95007.45409972, 101619.39692484, 112291.91587149,  
119468.78940319, 85711.94738019, 85921.5184041 , 100987.34882941,  
110468.3576032 , 119646.4340405 , 118642.95176402, 97776.77548477,  
108909.55245905, 93792.76159853, 78179.10260106, 77611.70749989,  
107263.84910266, 109022.9949612 , 101154.20425548, 118001.07011618,  
118028.40850139, 88399.87083151, 85940.02887821, 113983.11033738,  
96215.40257307, 120577.58901235, 107222.2427457 , 83797.64941189,  
115203.32773672, 114339.314005 , 110377.33500813, 80458.16320253,  
90957.89496389, 103387.38448171, 96086.26765944, 113503.51734572,  
79126.36458386, 111935.97716345, 120943.06398503, 78136.71997706,  
107498.52468176, 115570.50772641, 102970.82877786, 98152.31455831,  
74824.17726884, 85741.92923964, 102417.2923683 , 110298.74808687,  
120583.97314104, 125260.67932662, 95786.88443089, 122674.97360788,  
99801.76948737, 85587.79624937, 81838.26604651, 85872.22973704,  
70727.06213166, 85555.8184576 , 97254.32351877, 119662.06298033,  
94109.17332429, 92132.82919816, 101477.12519175, 122646.01180145,  
75334.26410673, 105044.22572617, 110888.43776677, 101358.14966336,  
98991.83201276, 85696.37475182, 102833.82241235, 111340.31713977,  
85315.23833668, 113981.78092329, 116243.87247741, 106731.05966132,  
101102.86019058, 108725.98997509, 110967.34731271, 86583.47621561,  
112998.2164714 , 111046.07443779, 113343.394686 , 105158.77068044,  
105802.96927267, 93974.9217062 , 111808.73618242, 113970.98280074,  
81146.83265141, 125558.80694005, 109554.76304181, 112654.32520556,  
109667.80786369, 112018.42994764, 86616.77941071, 96512.99382848,  
118553.5661195 , 98788.55652518, 125949.79741025, 101259.48763242,  
101746.51796067, 120180.70220538, 99520.64218606, 116983.16169048,  
79619.03673797, 106263.41595975, 120348.45122362, 98904.0120999 ,  
84470.6977612 , 102565.94552217, 121313.60272238, 109917.42794207,  
113307.91038565, 116645.33908948, 101189.48221892, 116800.63852845,  
77964.62128799, 109481.49411806, 111224.73803654, 97337.69933293,  
104299.39878837, 95444.60708401, 79348.71840834, 119273.88800902,  
101700.9753426 , 84727.37394698, 76797.8487362 , 105777.9894324 ,  
71669.69763524, 116037.86741424, 117231.91529504, 84803.70942047,  
83554.36838342, 91087.74418161, 76160.3709157 , 82776.24308911,  
114205.46228407, 106778.4030486 , 112993.30829999, 108018.83259272,  
73297.52862119, 118070.12675085, 116871.44232059, 86672.58283369,  
94003.6320588 , 77022.9899686 , 121782.47733265, 85654.64997036,  
83754.82001786, 94701.37404022, 106677.41567456, 103716.09906876,  
101505.15404997, 115622.58283454, 86527.23679186, 118175.09963995,  
109282.43414825, 98158.22743865, 78918.98381084, 115431.62123071,  
118844.53266533, 103996.42102495, 107206.77219656, 102676.28552177,  
112473.19322744, 98900.77649903, 112731.05345492, 110393.9908749 ,  
121571.66401454, 110884.57565243, 85295.76698939, 97091.29927797,  
93208.65788526, 113083.39910563, 119471.64870047, 118953.68223915,  
111705.16135786, 102924.69448732, 102184.17178787, 108235.70391276,  
95137.68594298, 97049.01361587, 101034.65893996, 115529.39132852,  
107288.8193383 , 106719.94631823, 102690.03809483, 94892.37059854,  
107437.47276945, 80853.86831169, 104231.52987543, 117716.08602022,  
96122.38294147, 106278.67247456, 101931.00017917, 97439.6609542 ,  
95679.79979918, 96583.94371922, 121911.17595658, 98516.0873207 ])
```

```
In [25]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_svm, predict), 4))  
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_svm, predict), 4))  
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_svm, predict)), 4))  
print("(R^2) Score:", round(metrics.r2_score(y_test_svm, predict), 4))
```

```
print(f'Train Score : {model.score(x_train_svm, y_train_svm) * 100:.2f}% and Test Score : {model.score(x_test_svm, y_test_svm) * 100:.2f}%')
errors = abs(predict - y_test_svm)
mape = 100 * (errors / y_test_svm)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%')
```

Mean Absolute Error: 452.4374

Mean Squared Error: 347419.2481

Root Mean Squared Error: 589.4228

(R²) Score: 0.9982

Train Score : 99.99% and Test Score : 99.99% using Support Vector Regressor.

Accuracy: 99.55 %.

```
In [26]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_svm_BVSP.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_svm_BVSP.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_svm_BVSP.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_svm_BVSP.csv")
```

```
In [27]: oneyear_df_pred = pd.read_csv("one-year-predictions_svm_BVSP.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Support Vector Machine")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

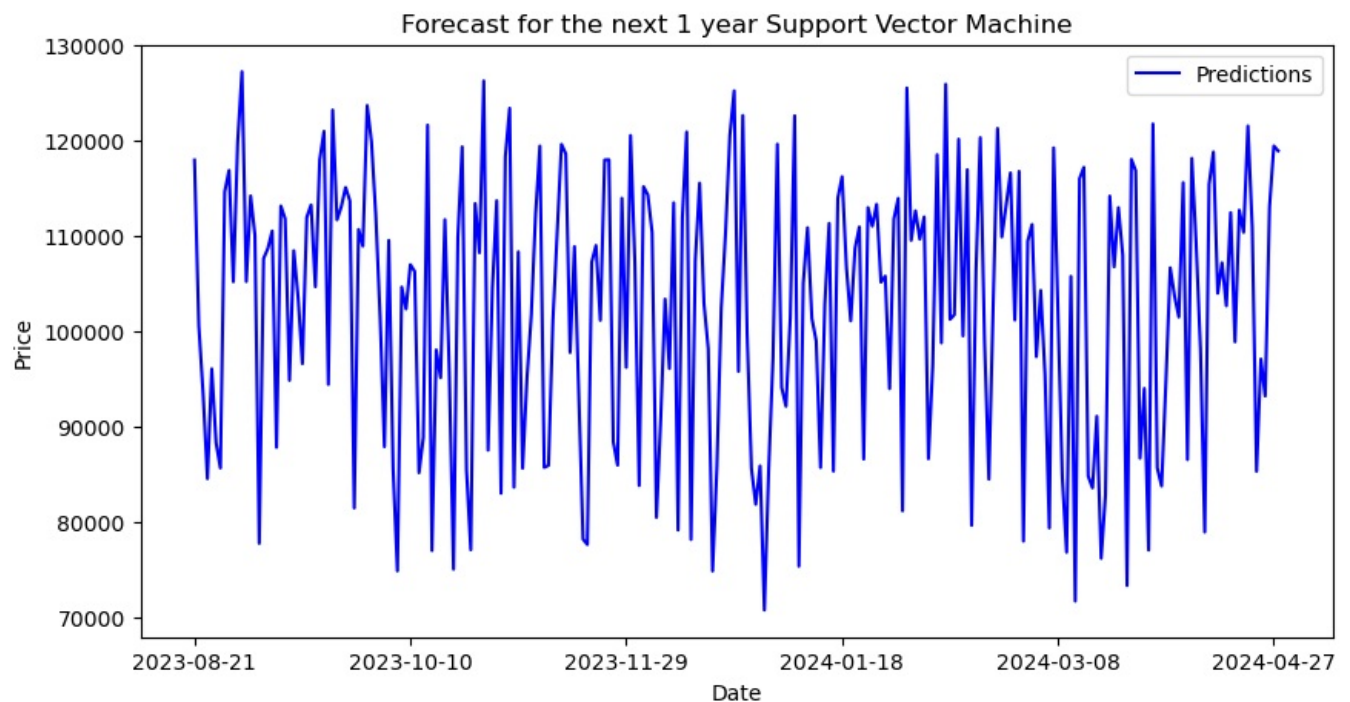
2023-12-31 70727.062132

Sell price and date

Predictions

Date

2023-09-01 127273.529749



```
In [28]: onemonth_df_pred = pd.read_csv("one-month-predictions_svm_BVSP.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
```

```

print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Support Vector Machi
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

```

```

Buy price and date
Predictions

```

```

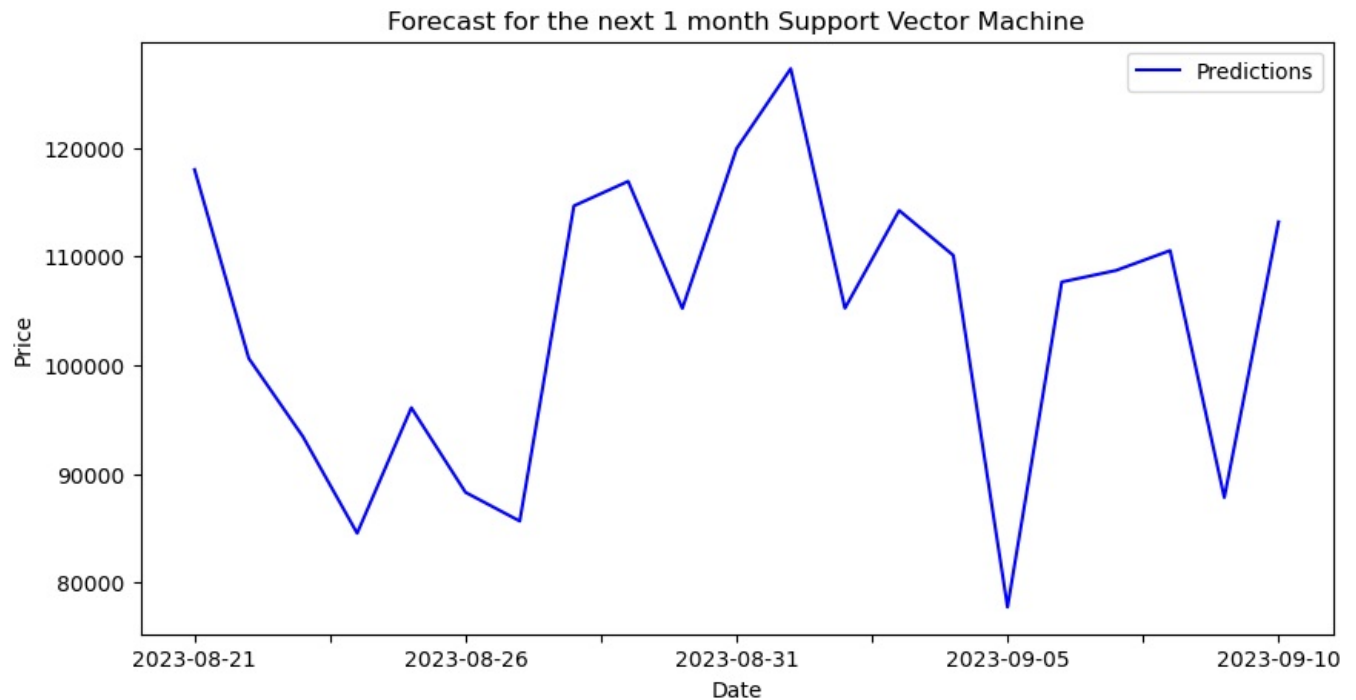
Date
2023-09-05  77728.489934
Sell price and date

```

```

Predictions
Date
2023-09-01  127273.529749

```



```

In [29]: fivedays_df_pred = pd.read_csv("five-days-predictions_svm_BVSP.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Support Vector Machin
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

```

```

Buy price and date
Predictions

```

```

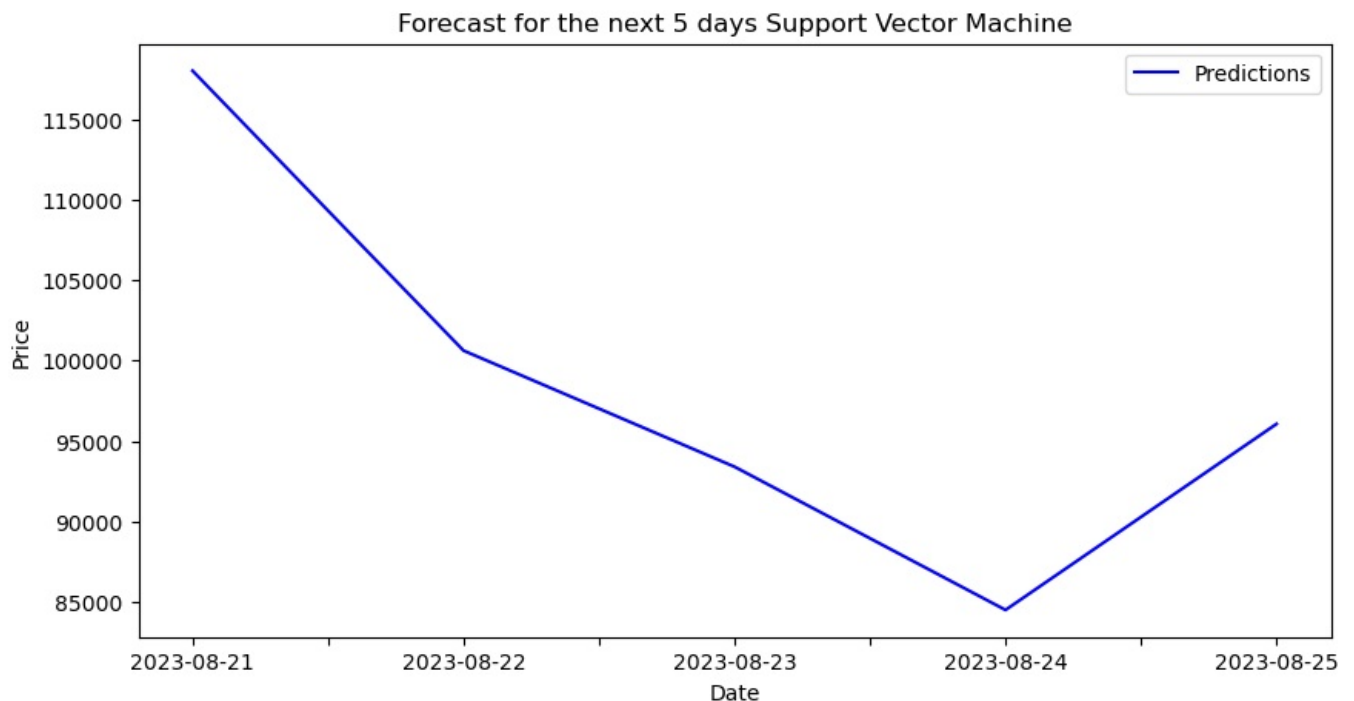
Date
2023-08-24  84525.851503
Sell price and date

```

```

Predictions
Date
2023-08-21  117991.409551

```



Decision Tree

```
In [30]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [31]: x_train_dt, x_test_dt, y_train_dt, y_test_dt = train_test_split(x, y, test_size=0.20, random_state=0)

In [32]: scale = StandardScaler()
x_train_dt = scale.fit_transform(x_train_dt)
x_test_dt = scale.transform(x_test_dt)

In [33]: # Define the parameter grid for hyperparameter tuning
param_grid = {
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Instantiate the Decision Tree model
dt_model = DecisionTreeRegressor()

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=dt_model, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train_dt, y_train_dt)

# Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)

{'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 5}
DecisionTreeRegressor(max_depth=10, min_samples_leaf=2, min_samples_split=5)

In [34]: # Retrain the best model on the full training set
best_model.fit(x_train_dt, y_train_dt)
```

```
# Make predictions
```

```
predict = best_model.predict(x_test_dt)
predict
```

```
Out[34]: array([[118641.      , 100296.      , 94614.33333333, 83843.      ,
 95938.      , 87918.5      , 85761.75      , 115060.      ,
116516.25    , 106008.66666667, 120238.5      , 126108.75      ,
104911.5     , 114901.      , 109700.5      , 78103.5      ,
108443.66666667, 109170.5      , 109926.33333333, 88556.75      ,
113035.5     , 111712.5      , 96747.      , 108966.66666667,
104273.33333333, 95687.      , 112170.5      , 112903.      ,
104735.75    , 117711.5      , 121806.      , 94275.      ,
122890.25    , 110238.66666667, 112750.      , 114430.33333333,
114310.      , 81266.      , 110571.33333333, 108358.75      ,
125227.      , 120446.5      , 112750.      , 100838.      ,
87918.5     , 108966.66666667, 85457.66666667, 75132.33333333,
104355.      , 102929.33333333, 107415.5      , 106008.66666667,
84904.33333333, 88943.      , 122088.66666667, 76260.5      ,
98564.75    , 95068.      , 112303.75      , 95595.33333333,
74682.33333333, 109875.      , 119398.25      , 85118.      ,
76260.5     , 113507.      , 108223.5      , 126108.75      ,
88556.75    , 104574.5      , 114088.      , 82571.33333333,
119215.      , 122622.25      , 83280.5      , 107735.6      ,
85619.      , 94614.33333333, 102266.75      , 112579.66666667,
119215.      , 85619.      , 85761.75      , 100542.25      ,
110421.75    , 120047.33333333, 118460.5      , 98044.      ,
109700.5     , 93628.25      , 77791.5      , 78589.66666667,
107357.5     , 109170.5      , 100267.66666667, 118410.      ,
117481.      , 88556.75      , 85504.5      , 113790.33333333,
96554.5     , 120695.25      , 106900.25      , 83843.      ,
114634.33333333, 115166.4      , 110051.66666667, 80252.33333333,
91924.5     , 102557.33333333, 96208.66666667, 114088.      ,
79554.      , 112303.75      , 122088.66666667, 78103.5      ,
106361.5     , 115944.      , 102266.75      , 97889.      ,
74682.33333333, 85504.5      , 102121.      , 110220.      ,
120838.5     , 125227.      , 95938.      , 122890.25      ,
98839.      , 85504.5      , 82571.33333333, 85761.75      ,
69789.25    , 85220.66666667, 97584.      , 119685.75      ,
92890.66666667, 91663.33333333, 102057.      , 122890.25      ,
74682.33333333, 102940.      , 111004.5      , 101015.33333333,
98839.      , 85220.66666667, 101453.      , 111487.      ,
85457.66666667, 114430.33333333, 116375.66666667, 107223.33333333,
101251.      , 107735.6      , 111004.5      , 86409.      ,
112303.75    , 109926.33333333, 112750.      , 104418.33333333,
105851.5     , 94488.      , 111099.      , 114634.33333333,
81266.      , 125551.66666667, 108794.5      , 112903.      ,
110128.5     , 110317.      , 86573.5      , 96208.66666667,
118041.25    , 98290.66666667, 126861.      , 99793.25      ,
102266.75    , 120238.5      , 99601.66666667, 117711.5      ,
79554.      , 107223.33333333, 120238.5      , 98696.      ,
84357.      , 101979.      , 121806.      , 110317.      ,
112903.      , 117602.66666667, 101251.      , 116673.5      ,
75942.5     , 108443.66666667, 110051.66666667, 97683.      ,
103928.75    , 95352.5      , 79148.      , 120047.33333333,
101917.75    , 84552.66666667, 77791.5      , 106445.      ,
70792.5     , 116233.      , 117481.      , 84552.66666667,
82571.33333333, 89922.      , 75372.      , 83108.75      ,
114983.75    , 106531.      , 112903.      , 108358.75      ,
72184.33333333, 119073.4      , 118041.25      , 86940.25      ,
93408.      , 76825.      , 121806.      , 85457.66666667,
83491.      , 94614.33333333, 105699.25      , 103872.5      ,
101858.      , 115541.25      , 86409.      , 118913.5      ,
109519.75    , 97257.      , 78934.25      , 115334.25      ,
119888.75    , 103673.      , 107670.66666667, 101803.25      ,
112475.25    , 98044.      , 112804.5      , 109926.33333333,
121266.      , 111004.5      , 84904.33333333, 96953.5      ,
93857.66666667, 112579.66666667, 119215.      , 119685.75      ,
111439.      , 103928.75      , 102266.75      , 108358.75      ,
94275.      , 97381.      , 100620.5      , 115334.25      ,
107529.25    , 106679.      , 102929.33333333, 95352.5      ,
108054.5     , 80426.5      , 104042.5      , 118762.33333333,
95938.      , 106008.66666667, 101201.      , 97683.      ,
95352.5     , 96091.25      , 121806.      , 97889.      ])
```

```
In [35]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_dt, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_dt, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_dt, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_dt, predict), 4))
print('Train Score : {model.score(x_train_dt, y_train_dt) * 100:.2f}% and Test Score : {model.score(x_test_dt,
errors = abs(predict - y_test_dt)
mape = 100 * (errors / y_test_dt)
accuracy = 100 - np.mean(mape)
```

```
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 80.7595

Mean Squared Error: 26078.9544

Root Mean Squared Error: 161.4898

(R²) Score: 0.9999

Train Score : 99.99% and Test Score : 99.99% using Decision Tree Regressor.

Accuracy: 99.92 %.

```
In [36]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_dt_BVSP.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_dt_BVSP.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_dt_BVSP.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_dt_BVSP.csv")
```

```
In [37]: oneyear_df_pred = pd.read_csv("one-year-predictions_dt_BVSP.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Decision Tree", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

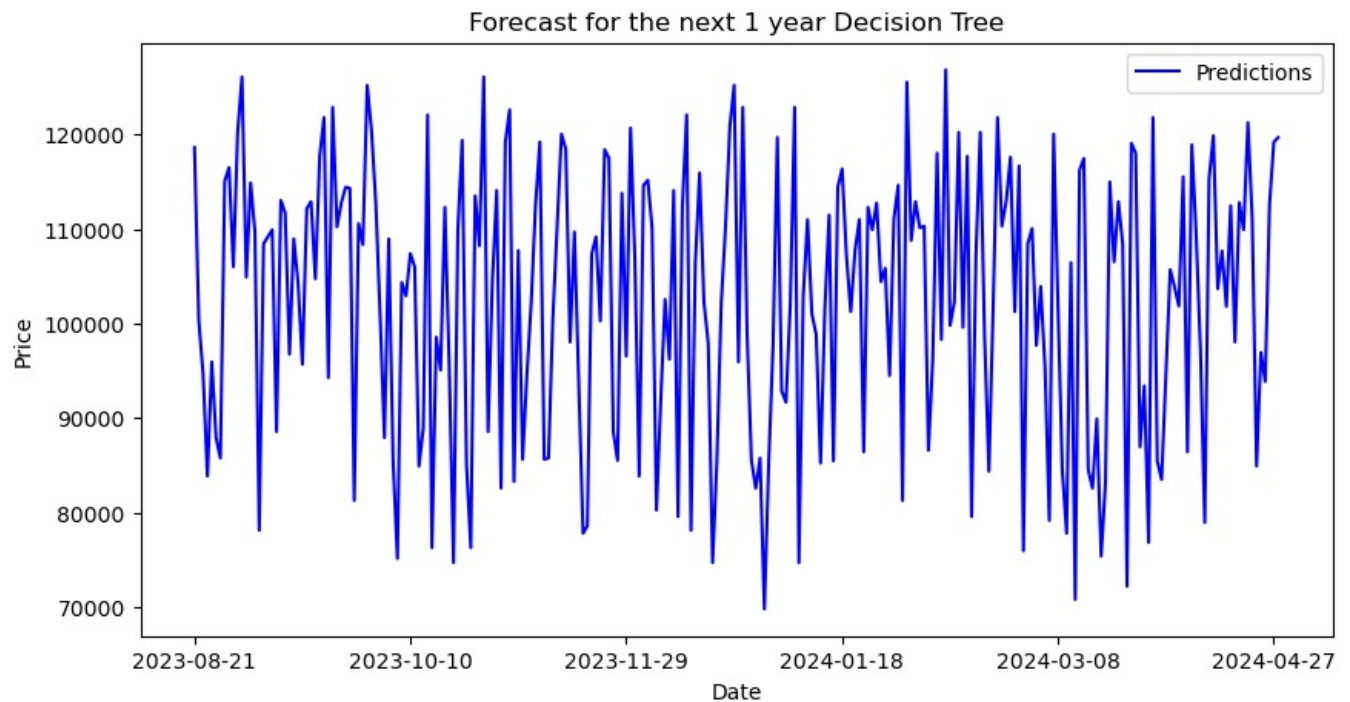
2023-12-31 69789.25

Sell price and date

Predictions

Date

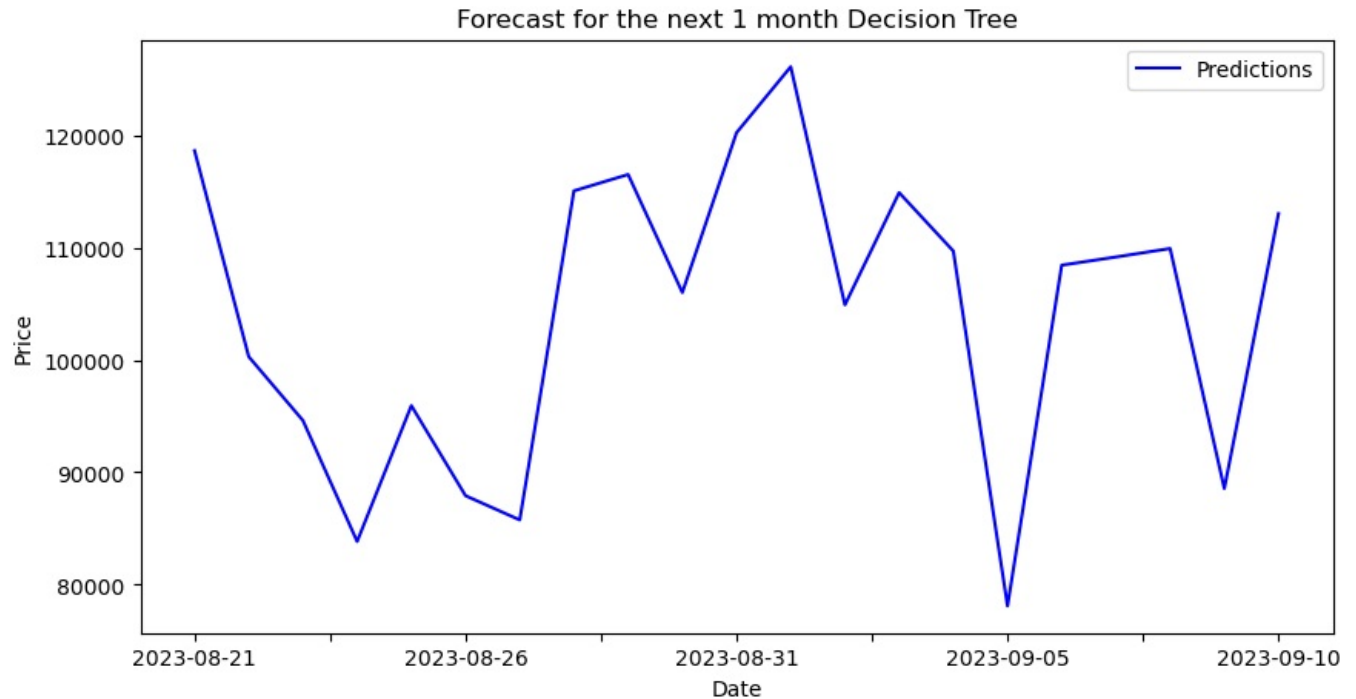
2024-02-11 126861.0



```
In [38]: onemonth_df_pred = pd.read_csv("one-month-predictions_dt_BVSP.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
```

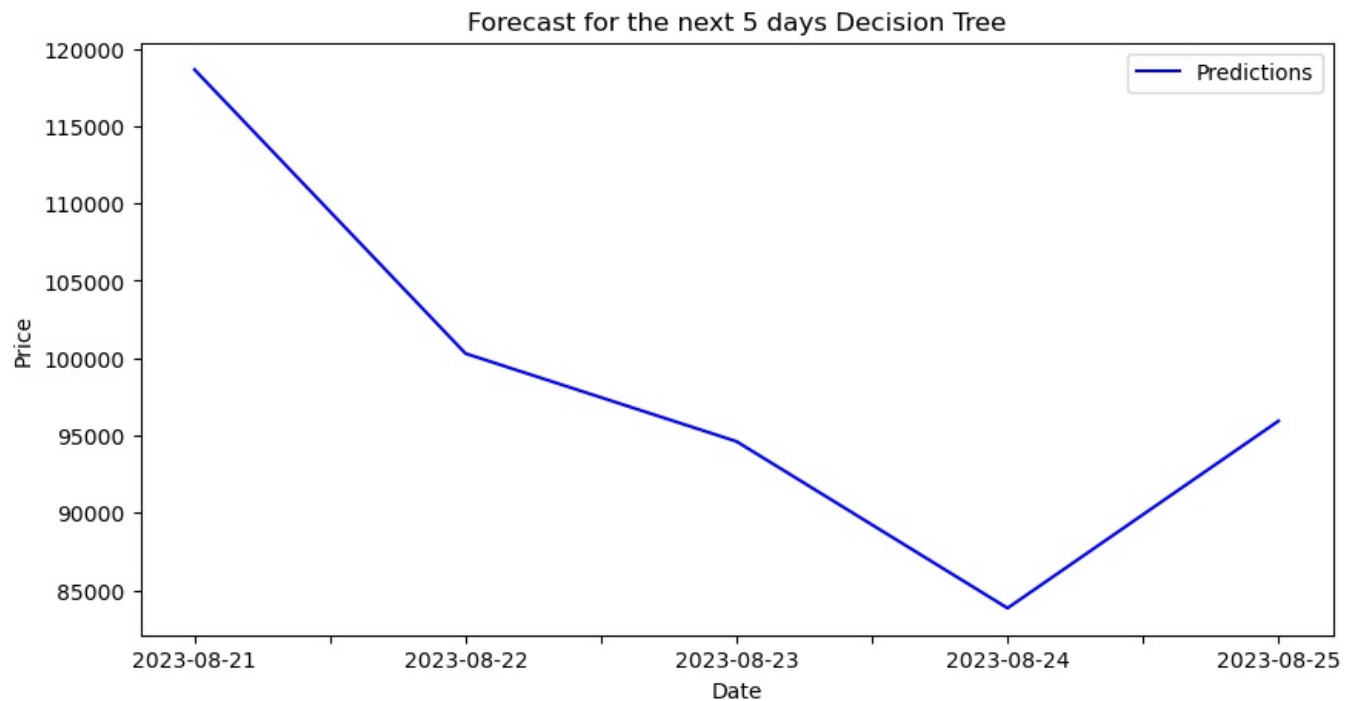
```
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Decision Tree", color="blue",
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
Predictions
Date
2023-09-05      78103.5
Sell price and date
Predictions
Date
2023-09-01     126108.75
```



```
In [39]: fivedays_df_pred = pd.read_csv("five-days-predictions_dt_BVSP.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Decision Tree", color="blue",
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
Predictions
Date
2023-08-24      83843.0
Sell price and date
Predictions
Date
2023-08-21     118641.0
```



Logistic Regression - ValueError: Unknown label type: 'continuous'

Multilayer Perceptron

```
In [40]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [41]: x_train_mlp, x_test_mlp, y_train_mlp, y_test_mlp = train_test_split(x, y, test_size=0.20, random_state=0)

In [42]: scale = StandardScaler()
x_train_mlp = scale.fit_transform(x_train_mlp)
x_test_mlp = scale.transform(x_test_mlp)

In [43]: # Define the parameter grid to search over
param_grid = {
    'hidden_layer_sizes': [(100,), (50, 50), (20, 20, 20)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive'],
}

# Create the MLP regressor
mlp = MLPRegressor(max_iter=1000)

rscv_mlp = RandomizedSearchCV(estimator=mlp, param_distributions=param_grid, cv=3, n_jobs=-1, verbose=2, n_iter=100)
rscv_fit_mlp = rscv_mlp.fit(x_train_mlp, y_train_mlp)

# Get the best parameters and best model
best_params = rscv_fit_mlp.best_params_
best_model = rscv_fit_mlp.best_estimator_

print(best_params)
```

```
print(best_model)
```

Fitting 3 folds for each of 10 candidates, totalling 30 fits

```
{'learning_rate': 'constant', 'hidden_layer_sizes': (20, 20, 20), 'alpha': 0.0001, 'activation': 'relu'}
```

```
MLPRegressor(hidden_layer_sizes=(20, 20, 20), max_iter=1000)
```

C:\Users\2227557\AppData\Local\anaconda3\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.

```
warnings.warn(
```

In [44]: *# Retrain the best model on the full training set*

```
best_model.fit(x_train_mlp, y_train_mlp)
```

Make predictions

```
predict = best_model.predict(x_test_mlp)
```

```
predict
```

C:\Users\2227557\AppData\Local\anaconda3\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.

```
warnings.warn(
```

```
Out[44]: array([[120042.82810354, 101747.44325631, 86386.34903078, 84642.24356257,
95095.1995545 , 88080.12677523, 83527.25115973, 114533.02809104,
112041.32922379, 101896.39189541, 121948.78993794, 126485.52237281,
107208.3761767 , 112099.40283531, 126060.43371038, 81268.52214593,
104936.72347833, 110091.9677617 , 105966.46883212, 85104.03391099,
110660.32189334, 114025.96848438, 90690.34957567, 111537.79543646,
102860.05286637, 93816.45812327, 116943.80369402, 116550.09114183,
106641.00616441, 120402.562448 , 124425.19048771, 93894.62468587,
125029.46883661, 103949.70830744, 115423.74828902, 115558.36485956,
112544.46715195, 80342.56592639, 109343.06039782, 113769.26080692,
123535.06180624, 123624.5952723 , 113073.58862001, 97974.70102871,
89850.42439982, 106508.79349265, 86970.87632789, 76586.59339268,
105739.19510021, 100895.72614184, 112407.04210217, 103225.7539589 ,
85309.58020798, 86062.9186102 , 119827.54784377, 76583.12630393,
96901.23992022, 96242.28824402, 113434.5467445 , 94282.67470465,
76074.17324982, 112623.31039474, 120679.27008418, 81016.73081258,
78124.17274441, 115498.22501447, 111511.19518774, 127732.50359334,
86450.65457198, 109145.71151202, 111212.5197788 , 82297.41588088,
114184.41265535, 117452.17363809, 85716.84491712, 107563.11492733,
85750.69137417, 92177.43625653, 101794.23793442, 129159.81966282,
120455.61506257, 84746.32324438, 85181.8975814 , 96907.49624809,
111266.18446056, 125357.27270502, 117539.45767952, 99222.43020924,
109447.18131527, 91180.40427575, 80089.05044279, 80754.36632179,
107269.95260143, 108576.64458852, 100310.00318615, 119312.5275142 ,
117208.54845693, 87324.31602322, 82802.64921853, 108151.36505271,
92732.3401117 , 117936.60543843, 106808.8621787 , 81810.20264434,
109549.77545918, 114780.26188454, 106186.31442274, 79511.16871373,
86881.95059329, 97379.97939915, 97516.67130968, 112498.27442712,
79523.57092179, 114038.80793908, 118945.95242051, 77968.41710402,
102308.39158079, 117195.85204371, 102812.72743089, 88677.90627528,
80718.13041782, 84620.63723126, 102451.08977102, 110689.70874527,
123252.72626929, 126467.04156437, 95389.99688281, 124400.17456635,
95845.39578843, 87607.30511792, 81612.40845367, 87229.78802001,
67622.42912474, 86763.79369611, 99744.50824697, 117424.9931908 ,
91922.42377809, 87600.96776941, 101716.7334581 , 123899.09309211,
77724.79805793, 92474.67647022, 114219.89910795, 98319.81580111,
95882.03810541, 85079.97481316, 102395.77766452, 112133.97373618,
84711.60426933, 114006.41143634, 118196.51873472, 106743.23642378,
101123.91134953, 105227.84101876, 115025.09749397, 88132.37072691,
105801.15150141, 105543.40350785, 105804.28347988, 105288.21845041,
107178.80733018, 95447.76149961, 111534.28279327, 112688.56194705,
83839.51131316, 127496.00082976, 111668.03413748, 114318.2139299 ,
113855.21657625, 102375.32026658, 87354.87497686, 94333.4464204 ,
119761.455399 , 99917.29815727, 126036.53163465, 93151.00971365,
102941.677653 , 116855.835605 , 97920.03067724, 118776.5138435 ,
81816.77363828, 105958.18116497, 122319.43237525, 100968.05907273,
84037.68361589, 105750.35738516, 120518.79949417, 108496.2811274 ,
113143.22849585, 119447.65336781, 99530.95021075, 118859.79201306,
72869.30110837, 108813.2751472 , 105987.80074267, 98830.20144245,
102193.45274207, 96641.78787073, 76474.0185261 , 119451.34927114,
98365.93939572, 87322.17819411, 70170.04272887, 108068.94552549,
72048.76788951, 118516.13444538, 120075.83013355, 85490.58746505,
80100.64444625, 85209.50821219, 79000.81085436, 81864.8635919 ,
113432.58935848, 105127.94078248, 115075.32725931, 107335.74359167,
69321.03893165, 116628.88809309, 113311.23186853, 89568.19304071,
88164.0944858 , 77136.92112064, 121679.23509507, 84386.73866215,
86245.73270437, 94763.1843739 , 105920.52747744, 101187.92315656,
100264.71870429, 114280.45651288, 87506.46437335, 119893.04333064,
112522.77188419, 96760.70936185, 73964.72137636, 113998.02685378,
121033.71794269, 107265.81304551, 105346.25772264, 105473.89750913,
114842.75119449, 95589.334655 , 110619.70961418, 110586.46283325,
117652.42157747, 113749.40124377, 86696.02198283, 97631.37697713,

96100.21071035, 110525.64029126, 118087.97078688, 117637.05185276,
104591.63088367, 103302.92182134, 101013.19333748, 108798.69350776,
94551.54539888, 96504.34562359, 99180.88160687, 112278.40527334,
111028.33364419, 104928.75918482, 100370.69277746, 92577.22488115,
103661.36874357, 81014.86672009, 105978.68194507, 117134.87170465,
94809.48145195, 105047.46595703, 98889.54493133, 98663.44312396,
90192.31120175, 92904.78776055, 119683.77715403, 99256.17309132]])
```

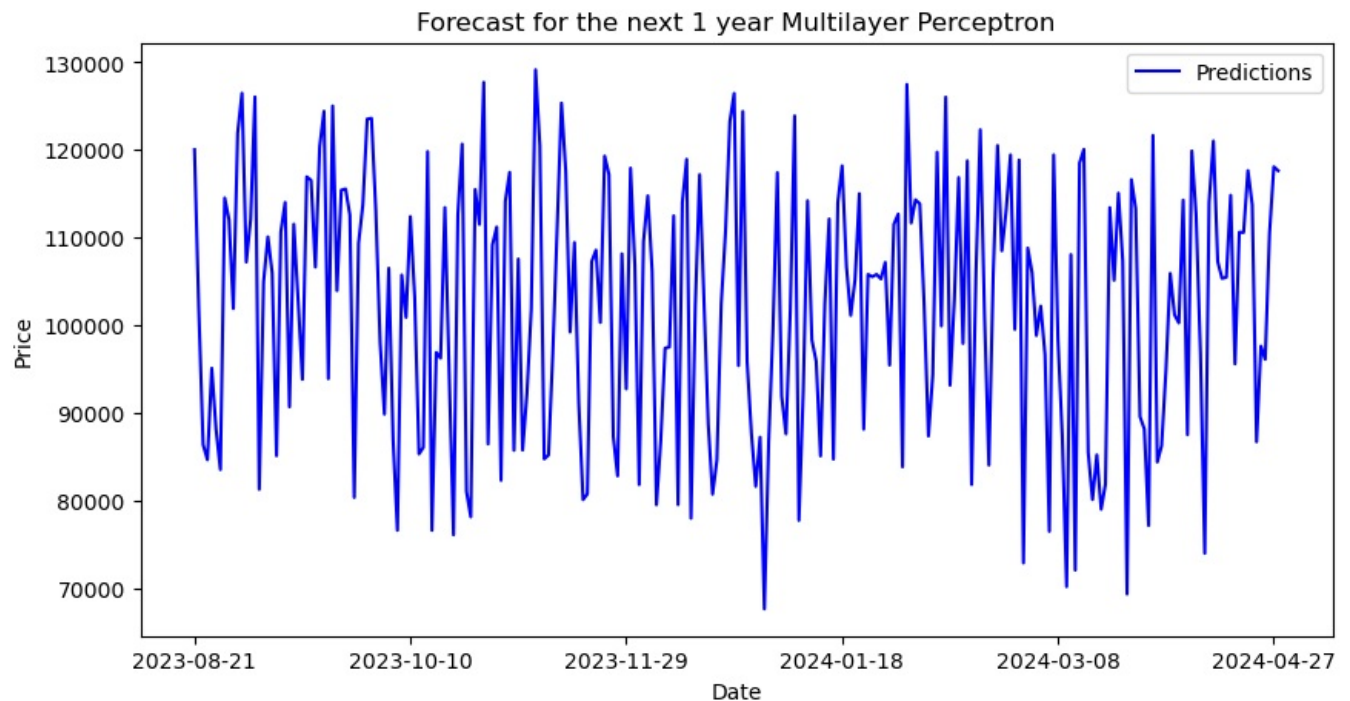
```
In [45]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_mlp, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_mlp, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_mlp, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_mlp, predict), 4))
print(f'Train Score : {model.score(x_train_mlp, y_train_mlp) * 100:.2f}% and Test Score : {model.score(x_test_mlp, y_test_mlp) * 100:.2f}%')
errors = abs(predict - y_test_mlp)
mape = 100 * (errors / y_test_mlp)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 2300.0157
Mean Squared Error: 9713186.4123
Root Mean Squared Error: 3116.5985
(R²) Score: 0.9486
Train Score : 99.99% and Test Score : 99.99% using Multilayer Perceptron Regressor.
Accuracy: 97.74 %.

```
In [46]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_mlp_BVSP.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_mlp_BVSP.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_mlp_BVSP.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_mlp_BVSP.csv")
```

```
In [47]: oneyear_df_pred = pd.read_csv("one-year-predictions_mlp_BVSP.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Multilayer Perceptron")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions
Date
2023-12-31 67622.429125
Sell price and date
Predictions
Date
2023-11-08 129159.819663



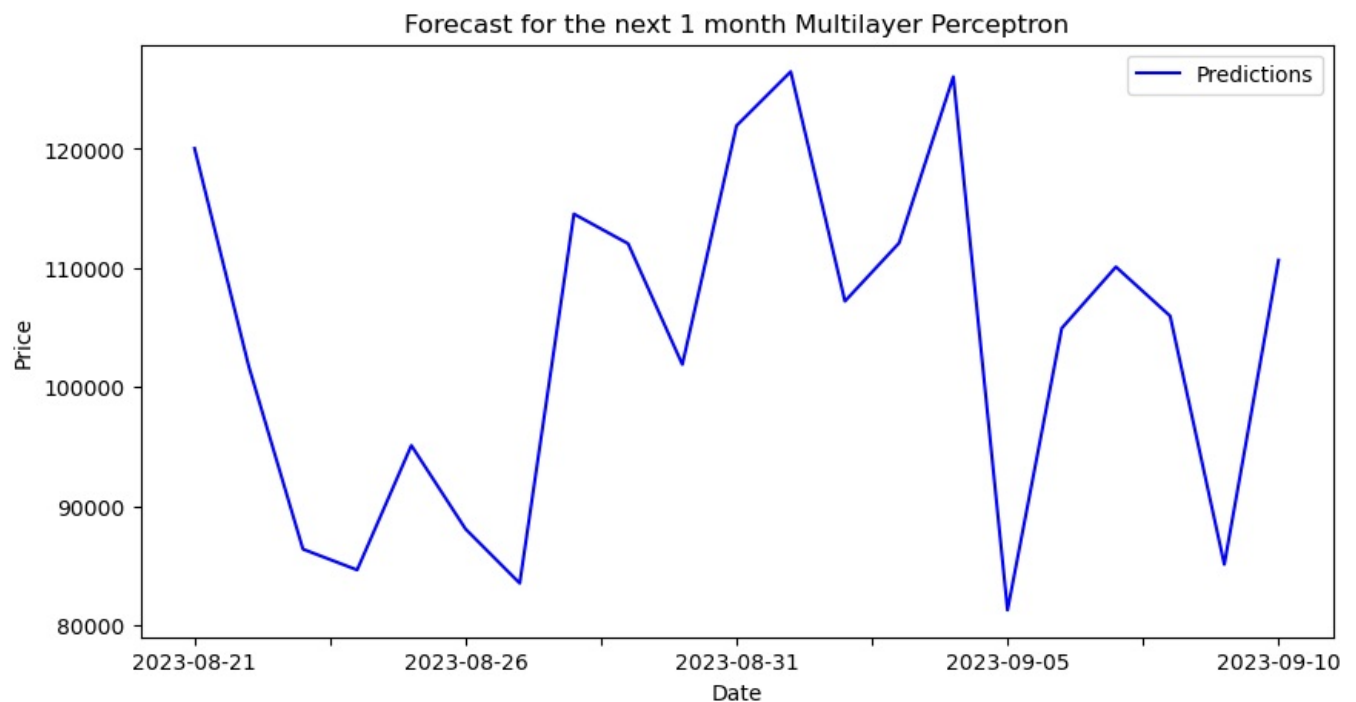
```
In [48]: onemonth_df_pred = pd.read_csv("one-month-predictions_mlp_BVSP.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Multilayer Perceptron")
plt.xlabel("Date")
```

```
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-09-05 81268.522146
Sell price and date

Predictions
Date
2023-09-01 126485.522373

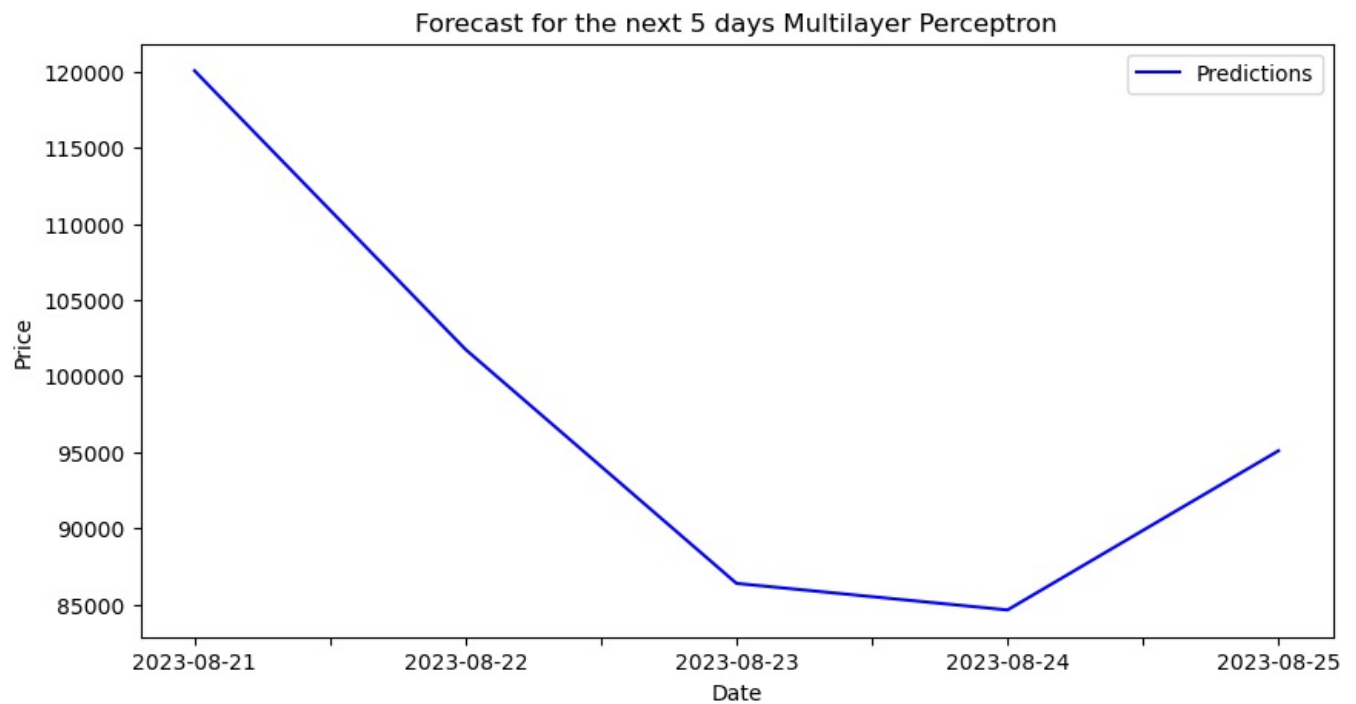


```
In [49]: fivedays_df_pred = pd.read_csv("five-days-predictions_mlp_BVSP.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Multilayer Perceptron")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-08-24 84642.243563
Sell price and date

Predictions
Date
2023-08-21 120042.828104



KNN

```
In [50]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [51]: x_train_knn, x_test_knn, y_train_knn, y_test_knn = train_test_split(x, y, test_size=0.20, random_state=0)

In [52]: scale = StandardScaler()
x_train_knn = scale.fit_transform(x_train_knn)
x_test_knn = scale.transform(x_test_knn)

In [53]: # Define the parameter grid to search over
param_grid = {
    'n_neighbors': range(1, 21), # Number of neighbors to consider
    'weights': ['uniform', 'distance'], # Weighting scheme for neighbors
    'p': [1, 2], # Power parameter for the Minkowski distance metric
}

# Create the KNN classifier
knn = KNeighborsRegressor(n_neighbors=5) # Set the number of neighbors (k) as 5

# Initialize the RandomizedSearchCV object
rscv_knn = RandomizedSearchCV(estimator=knn, param_distributions=param_grid, cv=5, n_jobs=-1, verbose=2, n_iter=100)

# Perform hyperparameter tuning
rscv_fit_knn = rscv_knn.fit(x_train_knn, y_train_knn)

# Get the best parameters and best model
best_params = rscv_fit_knn.best_params_
best_model = rscv_fit_knn.best_estimator_

```

```
print(best_params)
print(best_model)
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits
{'weights': 'distance', 'p': 1, 'n_neighbors': 6}
KNeighborsRegressor(n_neighbors=6, p=1, weights='distance')

```
In [54]: # Retrain the best model on the full training set
best_model.fit(x_train_knn, y_train_knn)

# Make predictions
predict = best_model.predict(x_test_knn)
predict
```

```
Out[54]: array([[118362.21727572, 100351.41761023, 94984.78682855, 84861.02191675,
96650.73076394, 87039.55929653, 85670.97415463, 114894.02060474,
116812.15029724, 105653.43134481, 120149.93651767, 127171.8152738 ,
105114.23866734, 114008.08938374, 109539.03011134, 78012.24843132,
107225.87686514, 108567.17952691, 111086.66093526, 88385.89170453,
113074.81901786, 111607.4896038 , 95620.56817498, 108905.81910767,
104380.45198803, 96191.77496998, 111431.06755042, 112700.73046719,
104289.91729058, 118153.07033874, 120623.28655647, 94872.92033688,
122733.58264075, 110927.63911273, 112754.76407066, 115174.77215338,
114224.62708976, 81160.29060671, 111183.75497385, 109734.66252972,
123096.30680274, 119385.05169659, 113165.41205103, 100910.65615064,
87325.8607841 , 108867.34687499, 85253.81443152, 75490.55396029,
102979.63850926, 102405.99904749, 107482.67902705, 106657.56005457,
85019.03250171, 87623.32657615, 120679.82364339, 76627.64928 ,
97736.66462797, 95020.00903719, 112410.59070465, 96334.4560102 ,
77431.80771671, 110019.69074942, 119505.45092844, 77859.75380844,
75891.88722595, 113296.48100635, 107842.45123177, 126613.97939429,
86808.79462318, 104159.88229198, 113647.66639471, 82744.82371484,
119258.70696439, 123322.54841972, 83712.93256919, 107648.29635562,
85071.70137447, 95500.56754036, 102518.4884285 , 112328.60219978,
119573.06568972, 85456.1952006 , 85635.97011492, 100439.23391995,
110853.37548408, 120019.45610963, 119396.31269398, 96899.9347511 ,
110032.11769694, 94190.74499656, 77889.58307976, 79143.81057175,
106627.19760261, 108827.30143813, 100940.61028134, 118052.04953598,
118063.4853429 , 87564.03888403, 85867.64150321, 114762.87830679,
95894.5159388 , 120261.67855285, 107163.33313228, 84735.13011667,
115411.27728866, 115405.68131558, 110037.12091204, 79690.13241162,
91839.69390775, 103991.1790699 , 96317.10364405, 113860.15277916,
78327.74391724, 111839.92638834, 120227.35154739, 77705.04692289,
107022.41597316, 116591.32132349, 102640.30617037, 99589.74728066,
75009.01098134, 85361.79155393, 101577.58316322, 111318.87692035,
121030.45465776, 125448.98723707, 95648.74165658, 121284.32579978,
99864.50486085, 85351.3937556 , 81904.5762936 , 85279.36710776,
69489.22561702, 85081.23389981, 97253.31137538, 119553.44138211,
93781.73656273, 93200.17520024, 102331.03211692, 122491.7338152 ,
76411.47786039, 106507.28445263, 111978.05871724, 100641.57588853,
99126.39765988, 85663.75357808, 102356.82593568, 111087.53015114,
84936.91659552, 113413.76306486, 116285.06459039, 107160.78709803,
100571.01766789, 108105.52491244, 110763.66544742, 85722.14499171,
113975.70348311, 110524.29837839, 113710.60257059, 104255.65076286,
104653.25163965, 94916.44341486, 111015.55166346, 114417.92053098,
81437.92261227, 126416.22865158, 109585.59503426, 112722.43895256,
109786.12534223, 111519.81336338, 86419.1373223 , 95550.77060031,
118274.99441167, 98396.7199905 , 125605.67813943, 101101.17309994,
102577.20024955, 118695.3795375 , 99693.412617 , 116917.56757436,
79320.99880688, 106806.24892351, 120440.04536856, 98905.31492603,
84660.73077097, 103344.29400142, 119325.03183093, 109711.74788755,
113153.39309844, 117023.77041062, 101124.23158735, 117069.77575063,
77779.98820789, 108718.87377173, 109732.36149548, 97295.30636906,
104652.26567012, 95254.1344134 , 78516.11393553, 119246.37987765,
101230.46773911, 83928.91926316, 78102.4640814 , 105998.88379926,
71095.49473254, 115966.16270699, 117772.22597453, 84607.87556201,
84474.50494138, 92122.8955747 , 76830.22017211, 82997.00053039,
114069.56914469, 106193.32098568, 113185.79031923, 108482.34671684,
73011.37816243, 117881.06372259, 115874.25452463, 86178.19092912,
93299.70840993, 76896.94210769, 120673.95997457, 85320.67425767,
85197.19664956, 95867.59632427, 106191.98401317, 103043.5627381 ,
102152.23830693, 115640.4468317 , 85904.91739058, 118455.0670093 ,
109187.28241201, 97228.01073604, 79211.66916481, 115657.28933542,
116667.33484031, 104185.23479527, 107990.42869188, 101887.40465185,
112642.78502637, 102923.71850201, 112677.71689141, 110405.95078927,
121776.26049271, 110696.0110819 , 84967.55742419, 96788.7328499 ,
94318.03862797, 113326.36298452, 118597.54801282, 119427.51400534,
112118.39777261, 104378.19660239, 102874.0645248 , 108309.00197759,
94858.39202375, 96661.05325808, 100888.73825218, 116246.28373531,
108114.62951242, 107067.62554717, 102660.05658616, 94975.72540837,
107882.7940193 , 79524.23186635, 103855.64546336, 117772.43089077,
96091.35815464, 106268.78764835, 102271.70392069, 97461.54497263,
95526.34733583, 98003.45597602, 122051.22304195, 98705.94581211])
```

```
In [55]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_knn, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_knn, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_knn, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_knn, predict), 4))
print(f'Train Score : {model.score(x_train_knn, y_train_knn) * 100:.2f}% and Test Score : {model.score(x_test_knn, y_test_knn) * 100:.2f}%')
errors = abs(predict - y_test_knn)
mape = 100 * (errors / y_test_knn)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 615.9551
Mean Squared Error: 904492.469
Root Mean Squared Error: 951.0481
(R²) Score: 0.9952
Train Score : 99.99% and Test Score : 99.99% using KNN Regressor.
Accuracy: 99.38 %.

```
In [56]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_knn_BVSP.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_knn_BVSP.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_knn_BVSP.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_knn_BVSP.csv")
```

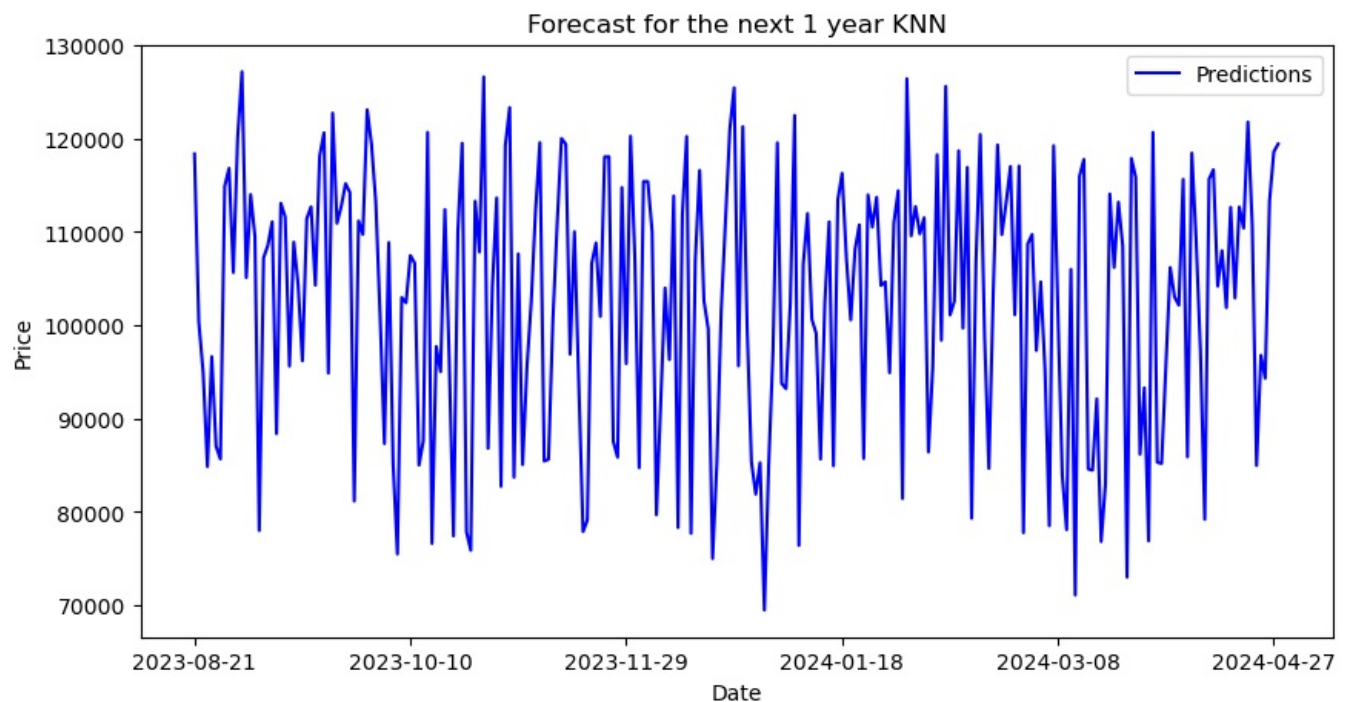
```
In [57]: oneyear_df_pred = pd.read_csv("one-year-predictions_knn_BVSP.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-12-31 69489.225617
Sell price and date

Predictions

Date
2023-09-01 127171.815274



```
In [58]: onemonth_df_pred = pd.read_csv("one-month-predictions_knn_BVSP.csv")
onemonth_df_pred.set_index("Date", inplace=True)
```

```

buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

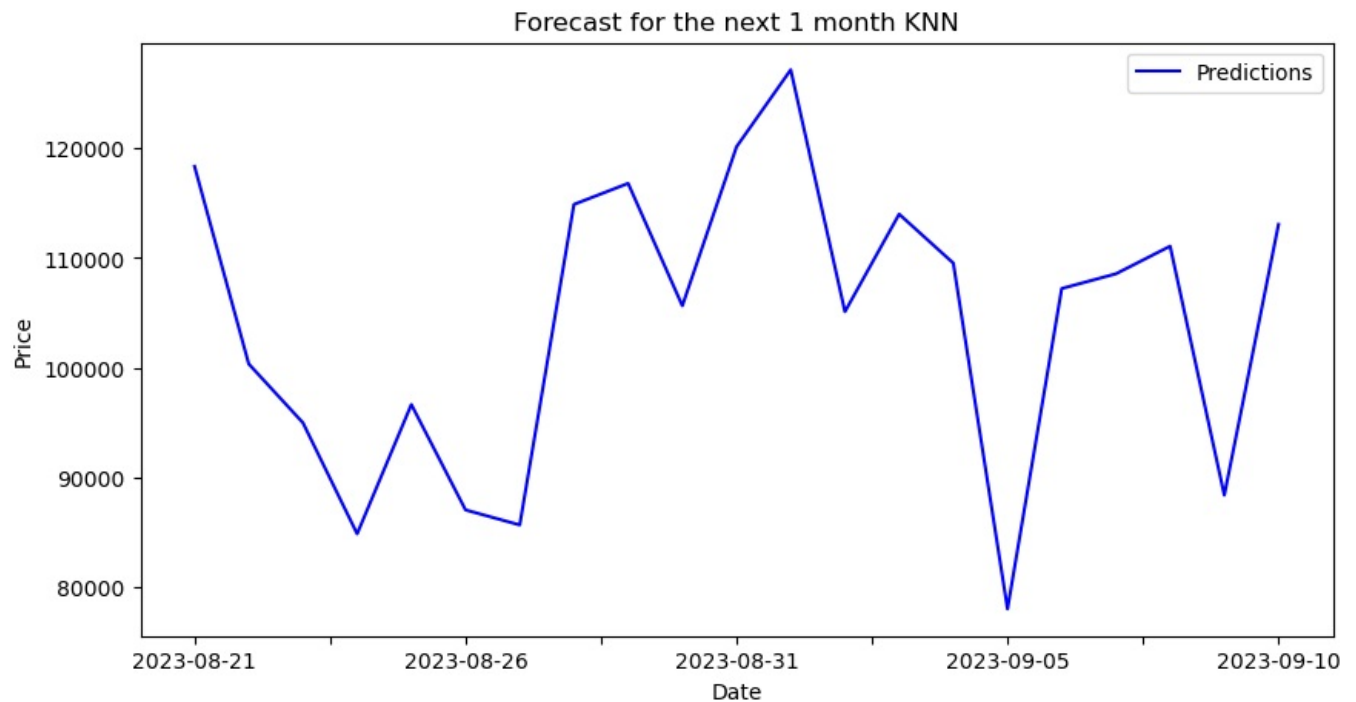
```

Buy price and date
Predictions

Date
2023-09-05 78012.248431

Sell price and date
Predictions

Date
2023-09-01 127171.815274



```

In [59]: fivedays_df_pred = pd.read_csv("five-days-predictions_knn_BVSP.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

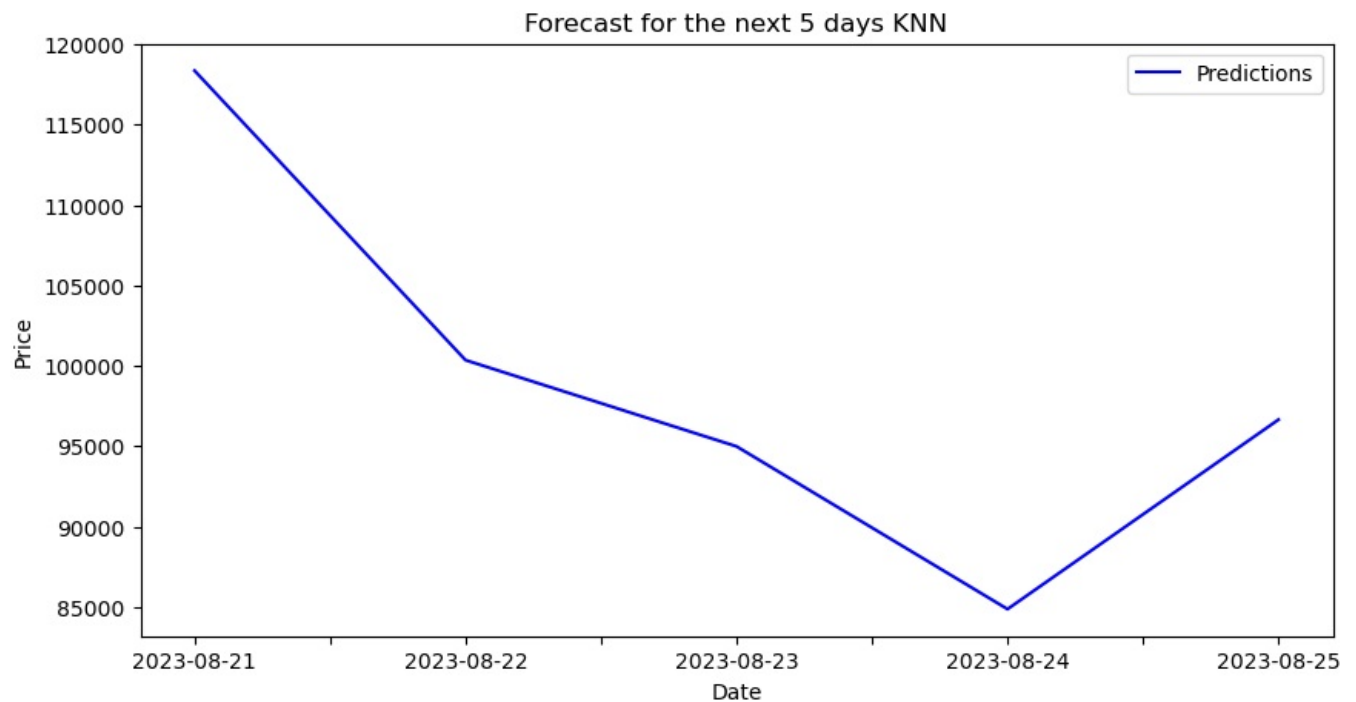
```

Buy price and date
Predictions

Date
2023-08-24 84861.021917

Sell price and date
Predictions

Date
2023-08-21 118362.217276



Clustering

```
In [60]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [61]: x_train_cls, x_test_cls, y_train_cls, y_test_cls = train_test_split(x, y, test_size=0.20, random_state=0)

In [62]: scale = StandardScaler()
x_train_cls = scale.fit_transform(x_train_cls)
x_test_cls = scale.transform(x_test_cls)

In [63]: # Create the SVR regressor
svr_regressor = SVR()

# Define the parameter grid to search over
param_grid = {
    'kernel': ['linear', 'poly', 'rbf'],
    'C': [0.1, 1.0, 10.0],
    'epsilon': [0.01, 0.1, 1.0]
}

# Initialize the GridSearchCV object
grid_search = GridSearchCV(estimator=svr_regressor, param_grid=param_grid, cv=3, scoring='neg_mean_squared_error')

# Perform hyperparameter tuning
grid_search.fit(x_train_cls, y_train_cls)

# Get the best hyperparameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)

{'C': 10.0, 'epsilon': 1.0, 'kernel': 'linear'}
SVR(C=10.0, epsilon=1.0, kernel='linear')

In [64]: # Retrain the best model on the full training set
best_model.fit(x_train_cls, y_train_cls)

# Make predictions
```

```
predict = best_model.predict(x_test_cls)
predict
```

```
Out[64]: array([[117991.55314155, 100622.23128608, 93412.13040143, 84524.88444769,
 96070.0936479 , 88284.20540771, 85644.05398088, 114647.78164053,
116904.87964588, 105210.33249695, 119909.52722295, 127274.1071128 ,
105230.03299372, 114228.78490902, 110094.12237136, 77727.64380328,
107643.79618158, 108703.12355357, 110536.69706289, 87810.48594296,
113176.48427701, 111814.48529599, 94852.10125915, 108474.96190532,
103554.66965066, 96601.75167867, 111978.80757415, 113283.00639975,
104685.61763314, 118001.18581328, 121005.1831169 , 94422.40544796,
123247.695015 , 111703.41087467, 113083.4726782 , 115113.82354607,
113696.04348859, 81441.08736699, 110706.12895776, 108951.09931398,
123722.23966365, 119882.48626496, 112148.4520021 , 101444.4640457 ,
 87882.88244602, 109558.31176755, 85606.45331835, 74833.55381615,
104652.04482215, 102348.96806533, 107007.88972387, 106284.94114364,
 85148.42974613, 88853.72410458, 121661.51375606, 76972.88417159,
 98054.02654212, 95129.18498059, 111734.71239429, 95977.49827036,
 75013.7093566 , 109858.66231904, 119383.18948047, 85426.27104783,
 77055.71152733, 113426.6364765 , 108254.58704128, 126310.18310333,
 87513.92005438, 104564.61103944, 113727.14653763, 82987.84756338,
118342.60325885, 123427.97564847, 83625.71260649, 108357.35311804,
 85616.56674702, 95006.61310744, 101620.82810104, 112294.33517968,
119469.43412857, 85711.97412936, 85920.36626881, 100987.36233338,
110468.38339956, 119646.54567308, 118642.81426927, 97775.95786444,
108908.789489 , 93791.71895922, 78177.55097626, 77609.14064555,
107262.88167021, 109023.28128824, 101153.72086839, 118001.87864383,
118028.5170242 , 88399.6382409 , 85939.47689795, 113983.61078617,
 96214.82693236, 120577.51368211, 107221.51798798, 83796.74111361,
115202.92683887, 114339.17924578, 110376.37846114, 80456.62371603,
 90956.26361313, 103387.39677817, 96085.54145211, 113503.67258081,
 79125.91095325, 111936.47672176, 120944.12959186, 78135.00536834,
107497.98642313, 115571.06673263, 102970.31158534, 98153.45888961,
 74823.67594635, 85741.03058921, 102416.75489108, 110299.29160869,
120584.18452647, 125262.07093135, 95787.22804941, 122676.25019237,
 99800.95103032, 85587.27406754, 81836.5858804 , 85870.93052842,
 70724.17559986, 85554.63182495, 97254.02707142, 119661.98912171,
 94107.82187531, 92131.9676506 , 101476.34590568, 122646.21386529,
 75333.10119497, 105043.18230991, 110888.97365345, 101358.99594053,
 98992.37395164, 85695.38522993, 102832.63947132, 111341.30610631,
 85313.65834 , 113981.86262495, 116244.65441961, 106731.80851673,
101101.73195755, 108725.29578819, 110967.19648225, 86582.45385444,
112997.4422261 , 111045.03880158, 113343.12431743, 105157.99981597,
105802.71562293, 93973.92738189, 111808.31477067, 113969.98582473,
 81145.2167794 , 125559.87241846, 109554.26035928, 112653.48024344,
109667.32275237, 112017.48516689, 86615.50060094, 96511.90909716,
118553.8760271 , 98787.66097772, 125951.18811748, 101258.13885635,
101745.9232298 , 120180.91937126, 99520.75489002, 116983.66944264,
 79617.79469691, 106262.86199183, 120348.33405194, 98903.51703469,
 84469.44764476, 102565.51998778, 121313.7999405 , 109916.93668568,
113307.53893253, 116643.47828744, 101188.57482278, 116800.9197703 ,
 77962.13091412, 109480.24134275, 111224.67628832, 97336.76024844,
104299.89130826, 95445.45340752, 79347.08061885, 119274.98125523,
101701.43325451, 84726.99694162, 76795.90462124, 105777.44657893,
 71668.13736902, 116037.35868178, 117232.04090865, 84803.12995949,
 83552.76958035, 91086.63106474, 76159.49164425, 82774.06096603,
114206.35517949, 106777.6233162 , 112993.12363324, 108019.49455374,
 73295.35098482, 118069.27542295, 116871.43999183, 86671.35764612,
 94003.94351838, 77021.39735922, 121782.64530944, 85653.69086438,
 83753.5695889 , 94700.83714415, 106676.58571173, 103715.78282649,
101504.81604172, 115623.18215573, 86526.52933073, 118176.28366432,
109281.89858487, 98157.20586913, 78917.70507692, 115431.73681739,
118844.09855318, 103996.37475222, 107207.79101316, 102675.3933636 ,
112472.46419409, 98901.62260398, 112730.68198816, 110393.74374828,
121572.03008374, 110884.23886221, 85295.07563667, 97090.73306801,
 93207.71616485, 113083.92000345, 119472.70898802, 118955.35652013,
111705.18768834, 102923.16006255, 102183.59139446, 108235.4748326 ,
 95136.47514383, 97048.10282419, 101034.44254109, 115529.51908812,
107288.91070419, 106720.63938455, 102690.39964906, 94891.83643935,
107436.48258423, 80852.23120895, 104231.72145005, 117716.41133835,
 96121.66095233, 106278.67232836, 101930.26073576, 97438.84242304,
 95678.80331267, 96583.59128754, 121912.09380005, 98515.29292296]])
```

```
In [65]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_cls, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_cls, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_cls, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_cls, predict), 4))
print(f'Train Score : {model.score(x_train_cls, y_train_cls) * 100:.2f}% and Test Score : {model.score(x_test_c
errors = abs(predict - y_test_cls)
mape = 100 * (errors / y_test_cls)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 452.2998
Mean Squared Error: 347159.7317
Root Mean Squared Error: 589.2026
(R²) Score: 0.9982
Train Score : 99.99% and Test Score : 99.99% using SVR Clustering Regressor.
Accuracy: 99.55 %.

```
In [66]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_cls_BVSP.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_cls_BVSP.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_cls_BVSP.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_cls_BVSP.csv")
```

```
In [67]: oneyear_df_pred = pd.read_csv("one-year-predictions_cls_BVSP.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Clustering", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

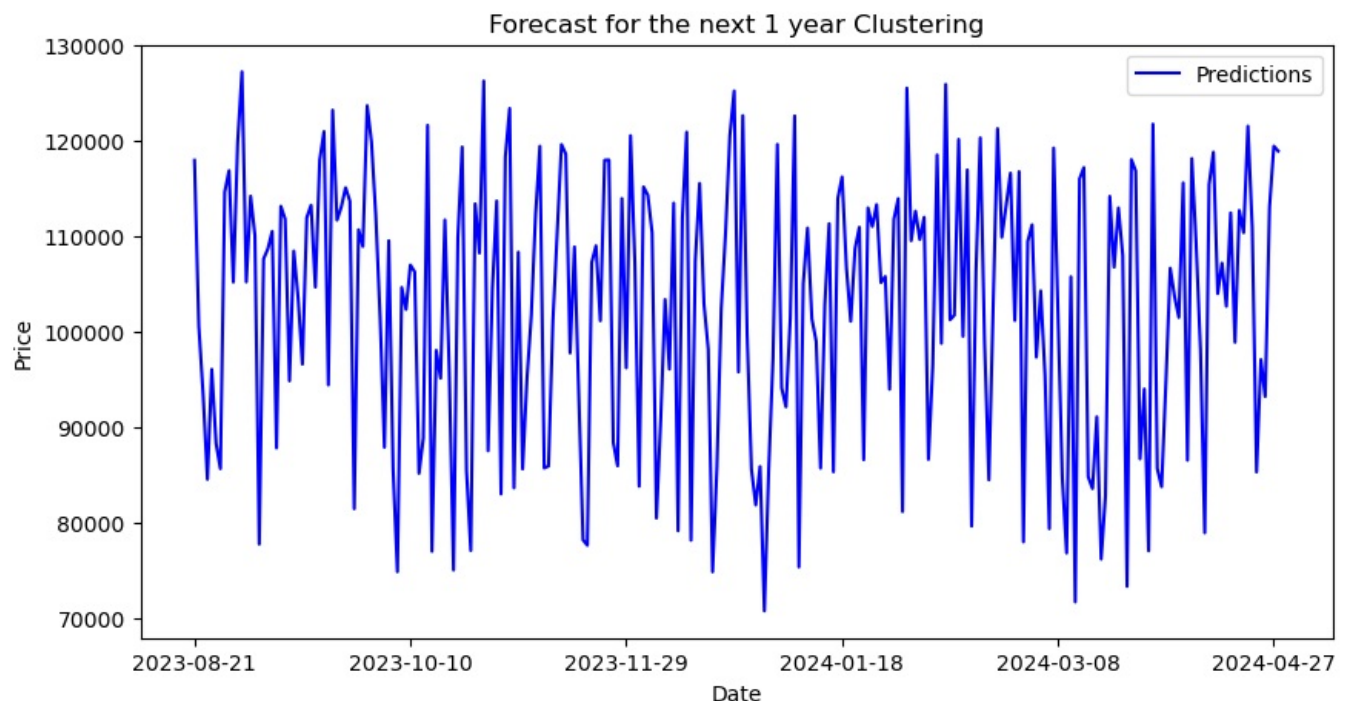
2023-12-31 70724.1756

Sell price and date

Predictions

Date

2023-09-01 127274.107113



```
In [68]: onemonth_df_pred = pd.read_csv("one-month-predictions_cls_BVSP.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Clustering", color="blue")
plt.xlabel("Date")
```

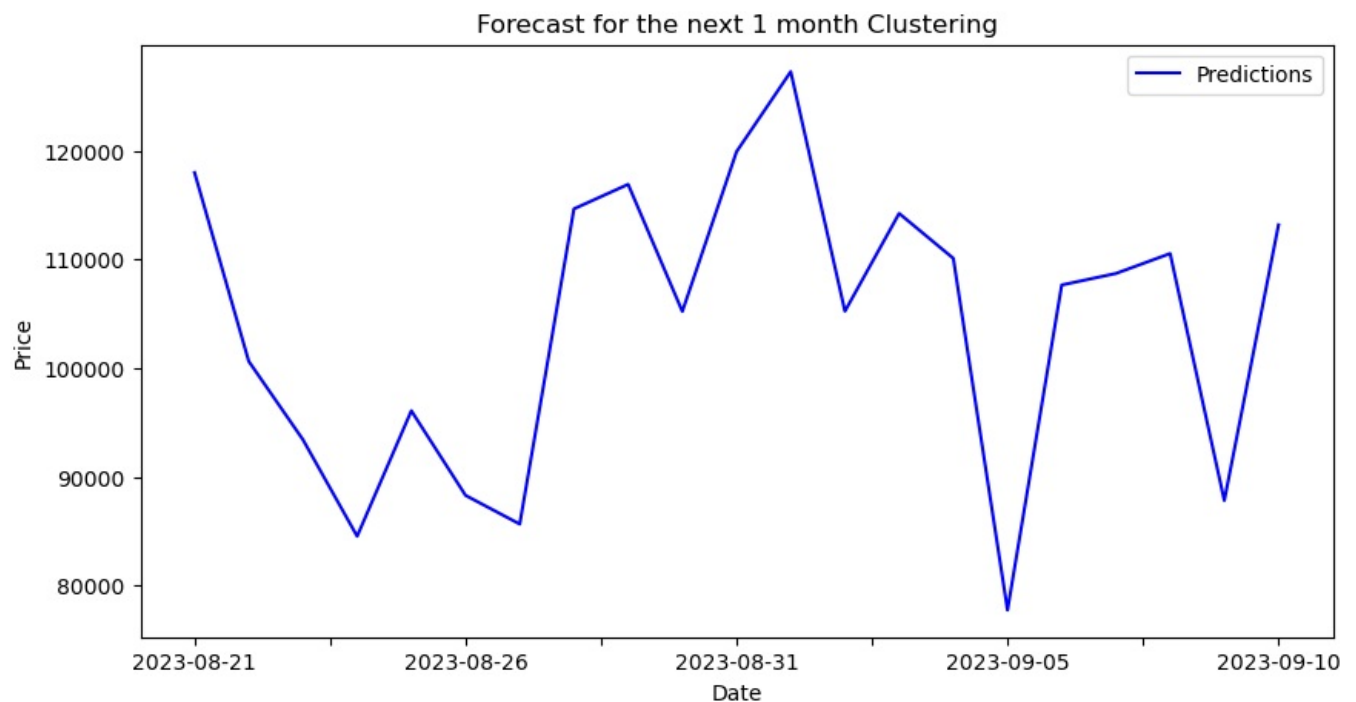
```
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-09-05 77727.643803

Sell price and date
Predictions

Date
2023-09-01 127274.107113



```
In [69]: fivedays_df_pred = pd.read_csv("five-days-predictions_cls_BVSP.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Clustering", color="b")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions

Date
2023-08-24 84524.884448

Sell price and date
Predictions

Date
2023-08-21 117991.553142

